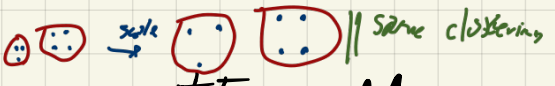
Returning to clustering, we still don't have a formalized structure for clustering. The attempts were numerous:

- KLEINBERG (2003): AXIOMATIZATION OF CLUSTERING FUNCTION

$$F((X,d)) = (C_1, \dots, C_k)$$ in general

|| (DESIRABLE) PROPERTIES OF F:



- SCALE INVARIANCE: F returns the same partition after scaling the data
- RICHNESS: $\forall C_i \in (C_1, \dots, C_k)\ \exists\ d$ (distance function) s.t. $F((X,d)) = (C_1, \dots, C_k)$ (so, distance function DETERMINES the output)
- CONSISTENCY: similar points will be clustered together and dissimilar points will be separated

We can prove an impossibility result:

IMPOSSIBILITY RESULT

So basically we don't have the guarantee that the cluster is perfect and we have an A and a cluster which is good in respect of which we are searching

$\nexists$ clustering function F that satisfies at the same time SCALE INVARIANCE, RICHNESS AND CONSISTENCY.

NEURAL NETWORKS

Initially, we approached to the optimization problem parametrically by minimizing the expected risk (or loss function).

$f: X \to Y$ $\quad S = \{(x_1, t_1), \dots, (x_m, t_m)\}$ $\quad t_i = f(x_i)$

H $\quad h(x) = h(x;\vartheta) = w_1 x_1 + w_2 x_2 + w_0$ (parametric again) $\quad \vartheta = \{w_1, w_2, w_0\}$

$L(h_S)$ is the loss function or ERM

Now, we will proceed by a parametrical approach as above, but $h(x)$ will not be a NEURAL NETWORK FUNCTION

FEED-FORWARD NEURAL NETWORKS (FNN) || the simplest one

↳ "Feed forward" means "single direction" there is no way to go back, given a vector $\vec{x}$ and also there is no dynamic memory of the process

OR

ARTIFICIAL NEURAL NETWORKS (ANN) ← General term

OR

MULTI-LAYER PERCEPTRON ← it generalization of the perceptron; ← Combining more perceptrons in a network

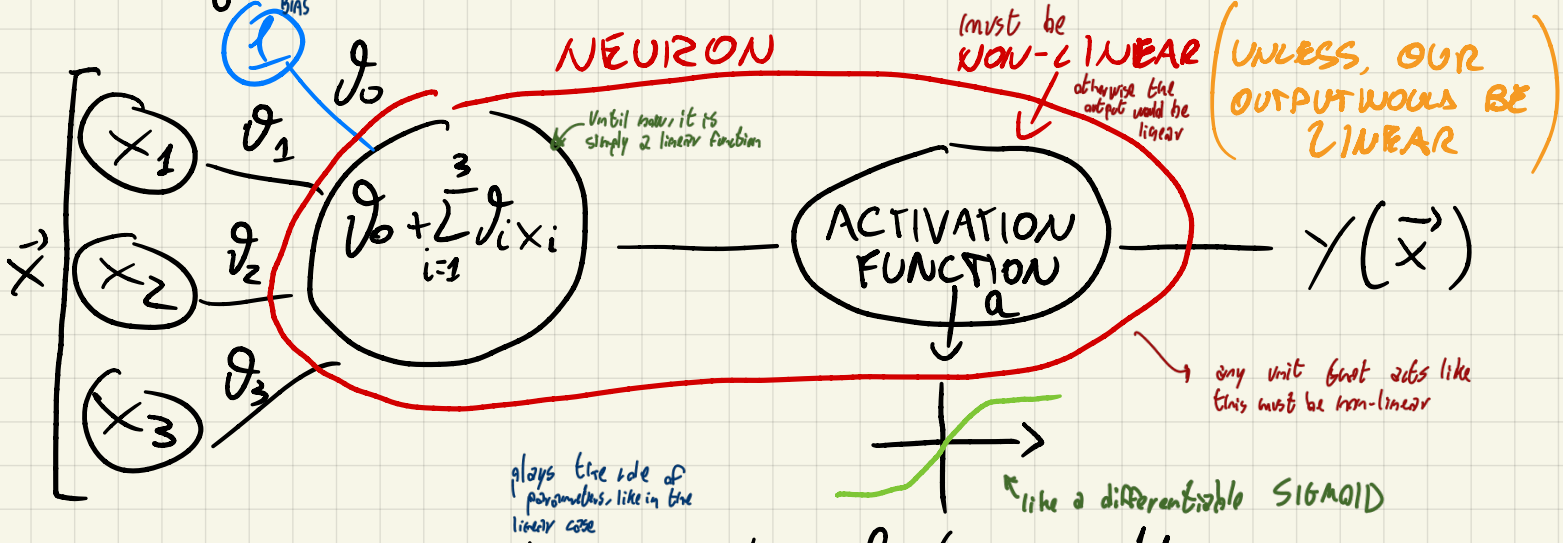
In previous models, it wasn't possible to fix or edit some variables during training (kernel SVM could, but with a lot of limitations). Here we will deal with more powerful but more complex models.

We do we need more powerful model? We can use kernel trick to model more than a linear perceptron. But has many problems:
1) Who gives us the kernel trick function?
2) With this models, you cannot model some stuff.
Neural Networks helps us. But finding the minimum of the expected loss it is harder because it is not convex anymore

SHALLOW FNN

(≠ DEEP)

The main element of NNs are neurons (human-based working elements). The model consists of two units:



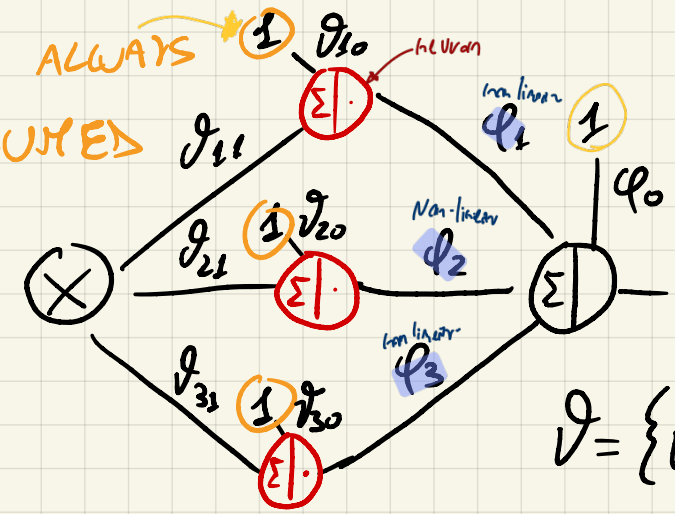
x_1, x_2, x_3 are input parameters, but 1 is the BIAS.

$$y(x_1, x_2, x_3) = a\left(\vartheta_0 + \sum_{i=1}^{3} \vartheta_i x_i\right)$$

activation function

Now, let's build a neural network, obtained by connecting neurons

BIAS IS ALWAYS ASSUMED



$\varphi_1, \varphi_2, \varphi_3$ are WEIGHTS FOR OUR FINAL OUTPUT

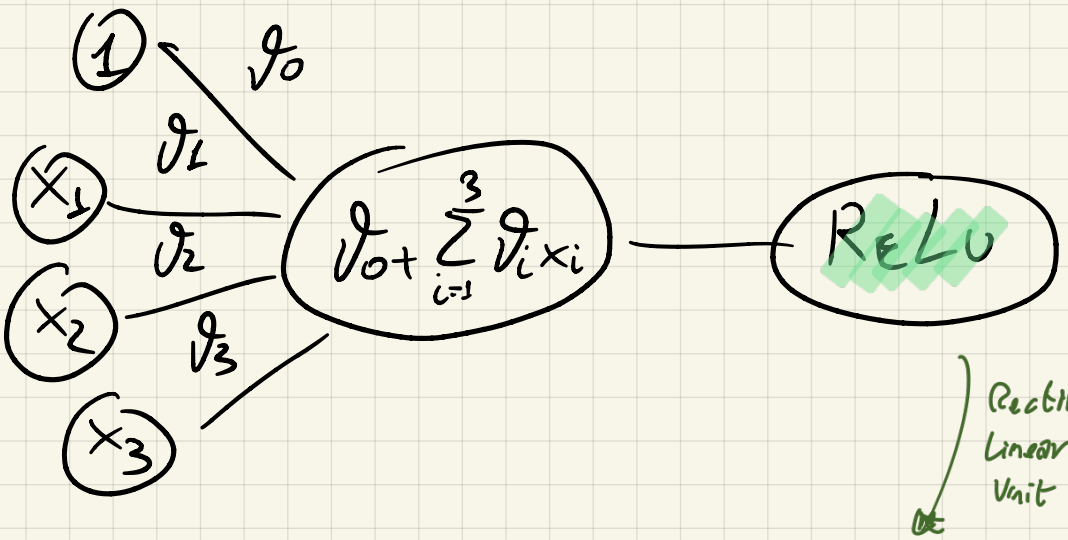
$$\vartheta = \{\vartheta_{10}, \vartheta_{11}, \vartheta_{20}, \vartheta_{21}, \vartheta_{30}, \vartheta_{31}, \varphi_1, \varphi_2, \varphi_3, \varphi_0\}$$

$$f(x_i;\vartheta) = \varphi_0 + \varphi_1 \cdot a(\vartheta_{10}+\vartheta_{11}x) + \varphi_2 \cdot a(\vartheta_{20}+\vartheta_{21}x) + \varphi_3 \cdot a(\vartheta_{30}+\vartheta_{31}x)$$

parametric function

FOR NOW, ACTIVATION FUNCTION IS UNIFORM FOR ALL NEURONS

Now, let's see what happens at the activation function (different from the sigmoid, but still non-linear):

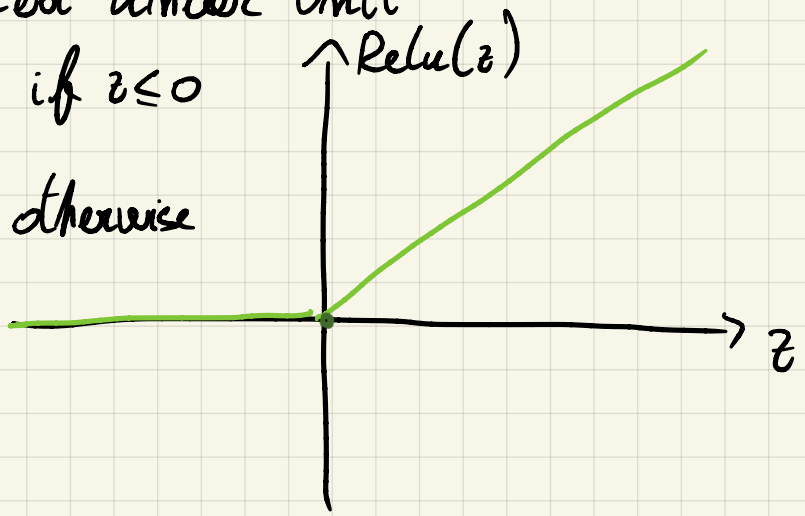


Rectified Linear Unit

RELU: Rectified linear Unit

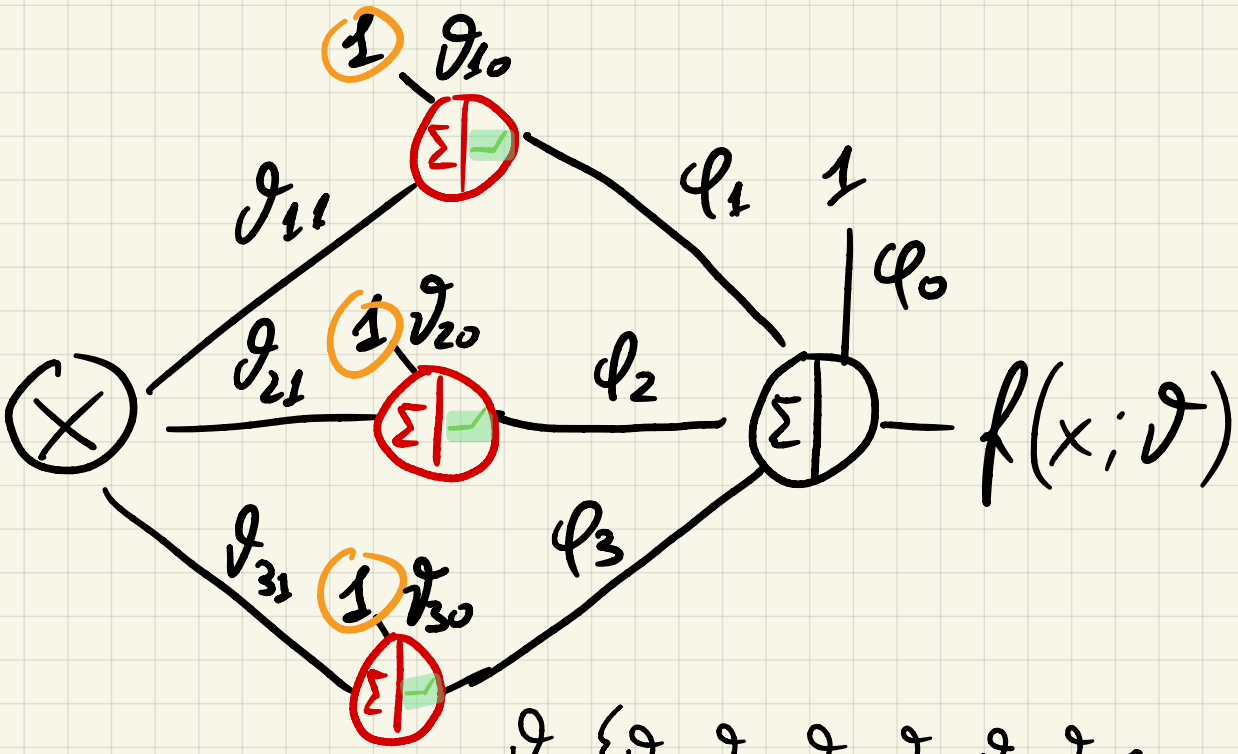
$$\text{Relu}(z) = \begin{cases} 0, & \text{if } z \leq 0 \\ z, & \text{otherwise} \end{cases}$$

becomes linear after zero



We don't have to use it but now we see it for an academic reason

Let's our NN with ReLu:



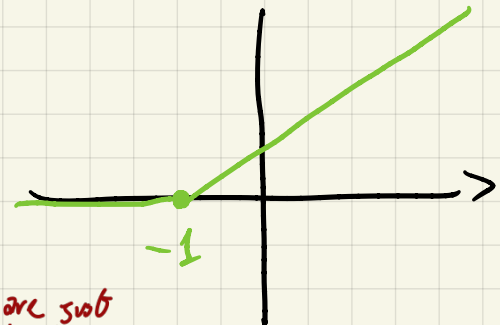
$$\vartheta = \{\vartheta_{10}, \vartheta_{11}, \vartheta_{20}, \vartheta_{21}, \vartheta_{30}, \vartheta_{31}, \varphi_1, \varphi_2, \varphi_3, \varphi_0\}$$

$$f(x_i;\vartheta) = \varphi_0 + \varphi_1 \cdot a(\vartheta_{10}+\vartheta_{11}x) + \varphi_2 \cdot a(\vartheta_{20}+\vartheta_{21}x) + \varphi_3 \cdot a(\vartheta_{30}+\vartheta_{31}x)$$

Let's apply $y = Relu(\vartheta_{10} + \vartheta_{11}x)$ (just the higher (on top) neuron) and see some cases:

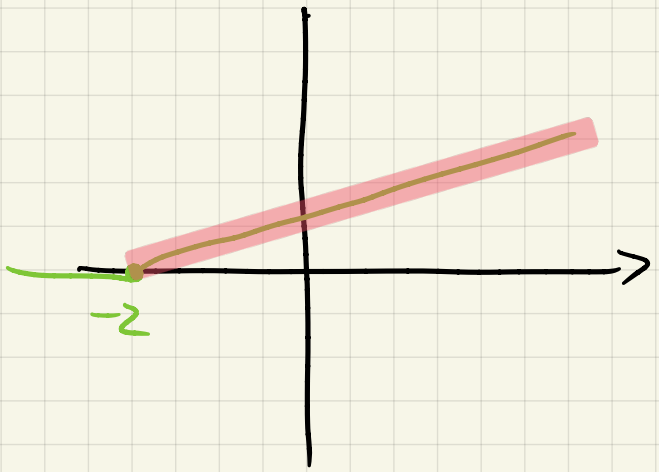
- $\vartheta_{10} = 0;\ \vartheta_{11} = 1$; $f(x;\vartheta) = Relu(x)$ ← Naive Relu

- $\vartheta_{10} = 1;\ \vartheta_{11} = 1$; $f(x;\vartheta) = Relu(1+x) = \begin{cases} 0 & \text{if } 1+x \le 0 \text{ i.e. } x \le -1 \\ x+1 & \text{else} \end{cases}$



- $\vartheta_{10} = 1;\ \vartheta_{11} = \frac{1}{2}$; $f(x;\vartheta) = Relu\left(1 + \frac{1}{2}x\right) = \begin{cases} 0 & \text{if } 1+\frac{1}{2}x \le 0 \text{ i.e. } x \le -2 \\ \frac{1}{2}x+1 & \text{else} \end{cases}$

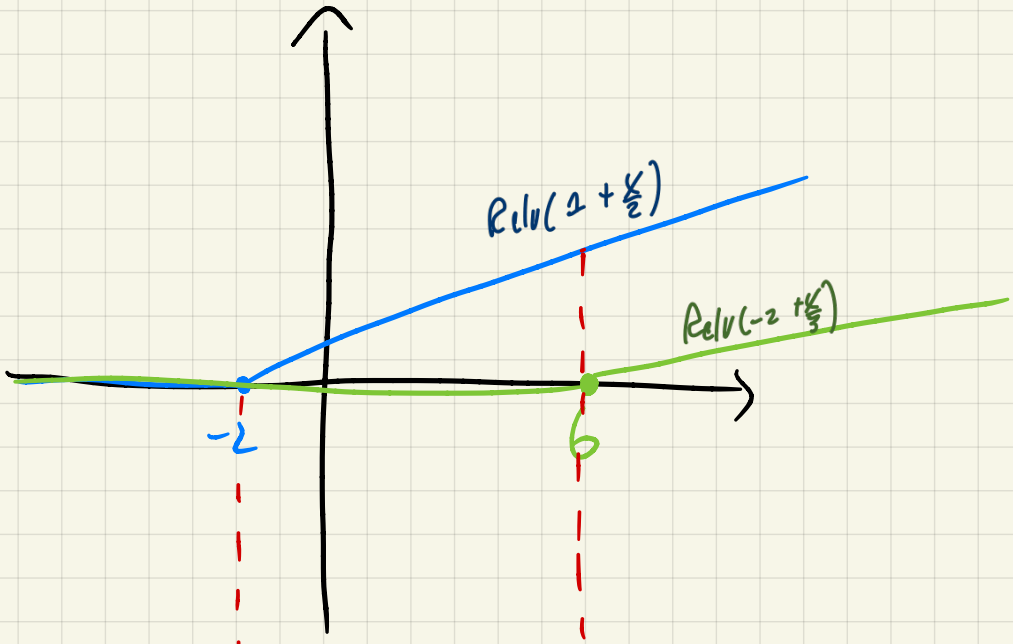
we are just changing the slope



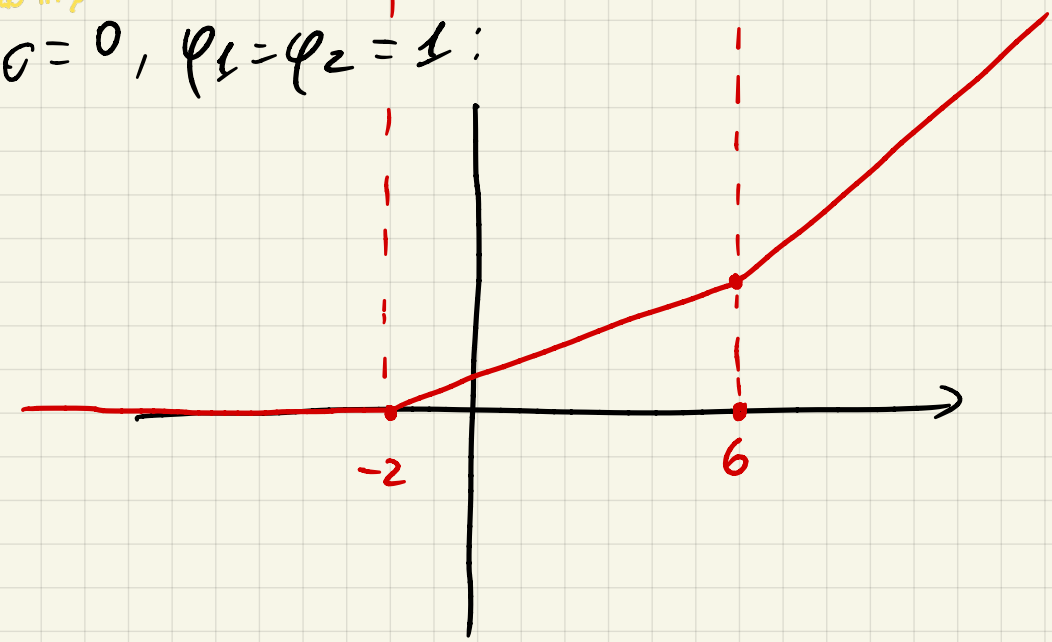
What if I combine the first 2 neurons' parameters?

- $\vartheta_{10} = 1$; $\vartheta_{11} = \frac{1}{2}$ $\rightarrow$ $\underline{\text{Relu}\left(1 + \frac{x}{2}\right)}$
- $\vartheta_{20} = -2$; $\vartheta_{21} = \frac{1}{3}$ $\rightarrow$ $\underline{\text{Relu}\left(-2 + \frac{x}{3}\right)}$

WE STILL DO NOT CONSIDER THE BIAS

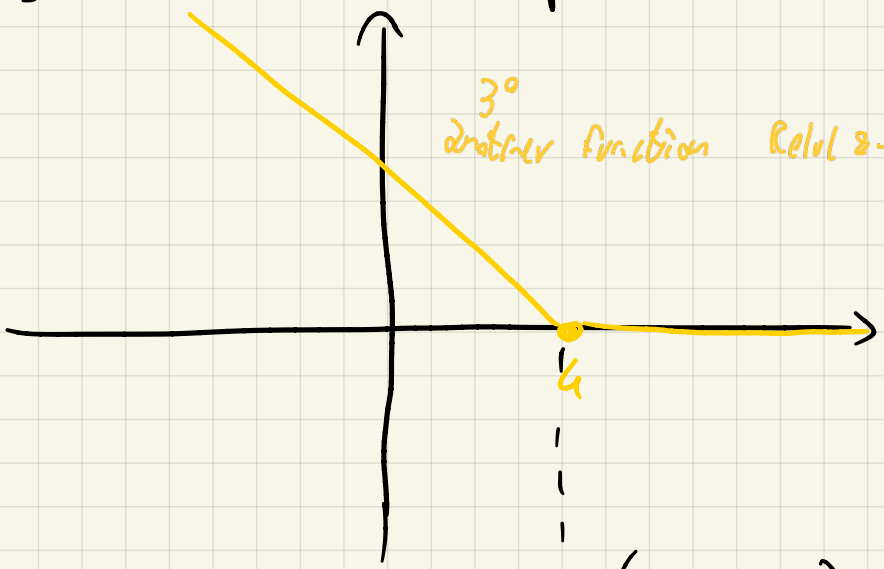


For bias in ϕ $\varphi_0 = 0$, $\varphi_1 = \varphi_2 = 1$:

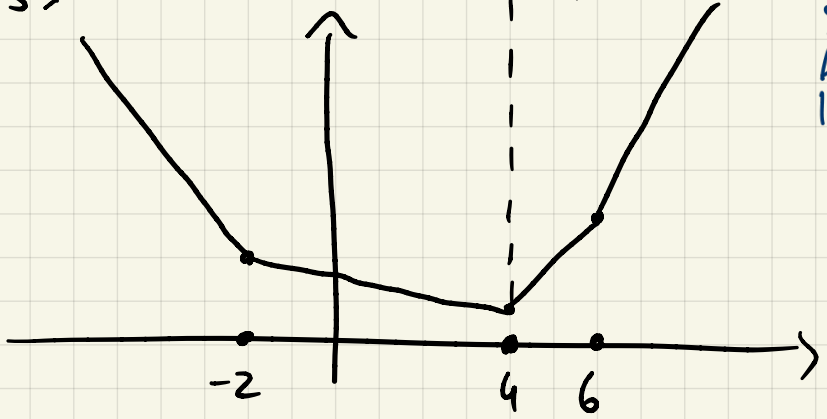


easy !!

Let's see what happens also with the 3rd neuron, in such a way that I want to produce this function:



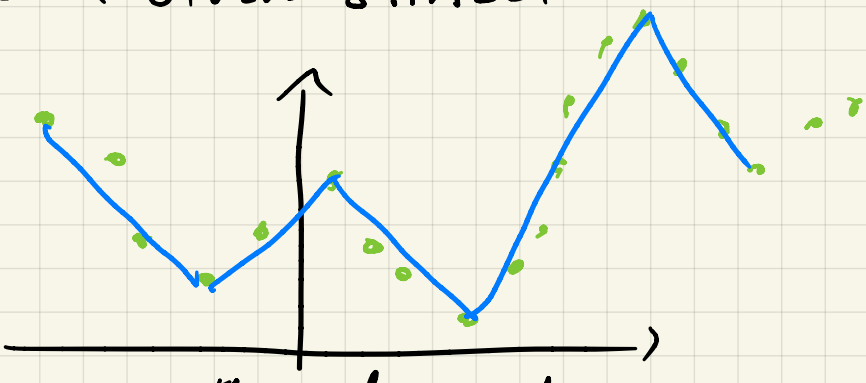
- $\vartheta_{30} = 8$; $\vartheta_{31} = -2$ → $Relu(8-2x)$



So, by using 3 units, we have produced a piecewise linear function

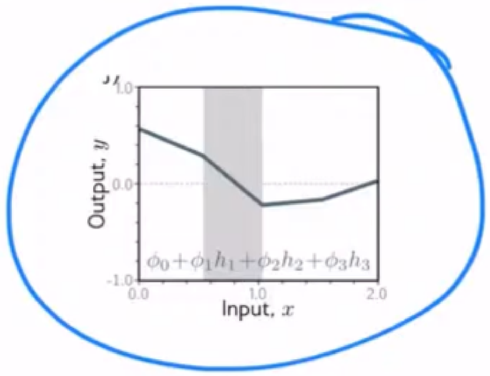
So, by "tuning" the parameters, we build a piecewise linear function by using a non linear one as input. But let's go back to our goal: we want to minimize our loss function for a given dataset.

EXAMPLE FOR A GIVEN DATASET



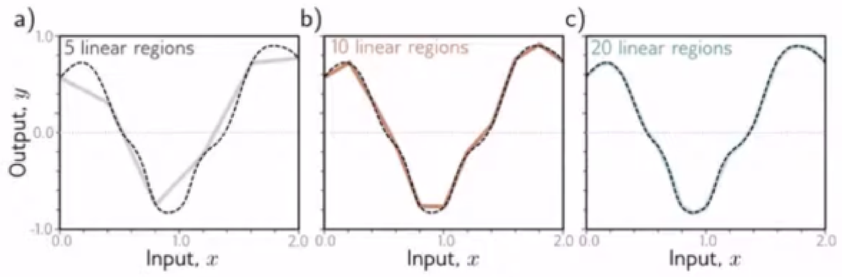
by combining more neurons, we can have even better

So, the intuition is that for a feature map, by building a number of units I can fit very well on our target.



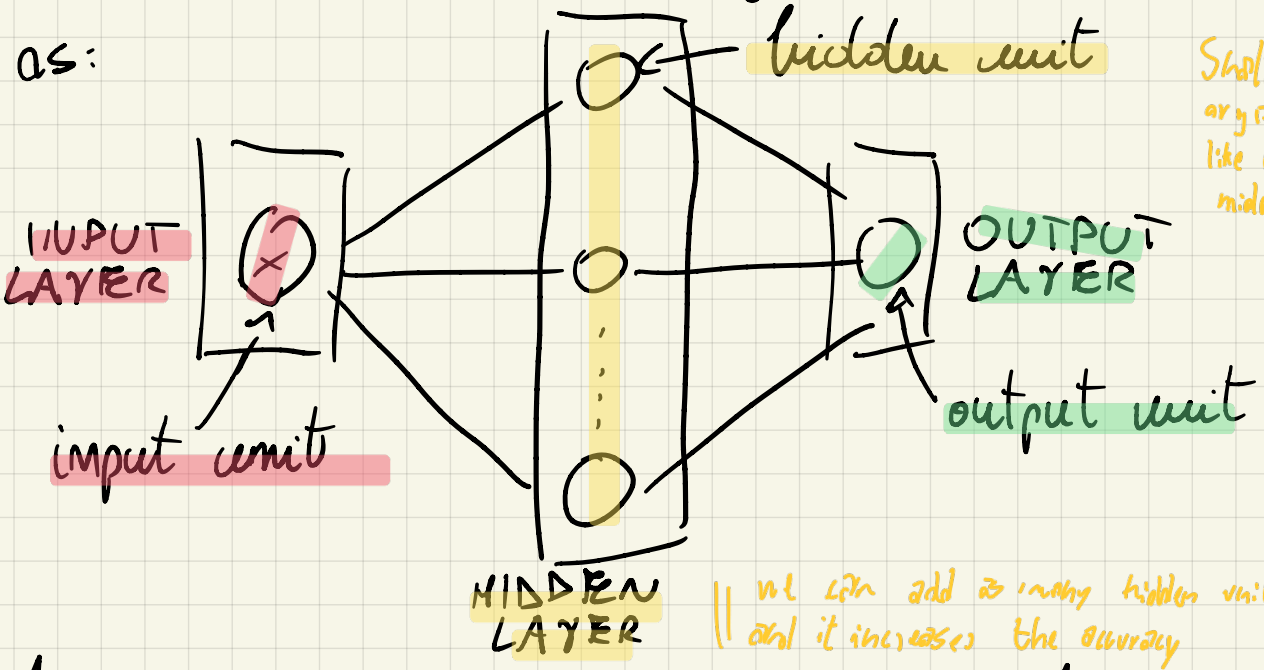
3 units: piecewise linear function with 4 linear regions

n units: piecewise linear function with $n+1$ linear regions



by increasing the number of breakpoints, it becomes even more precise

In our neural network build, our units are labelled as:



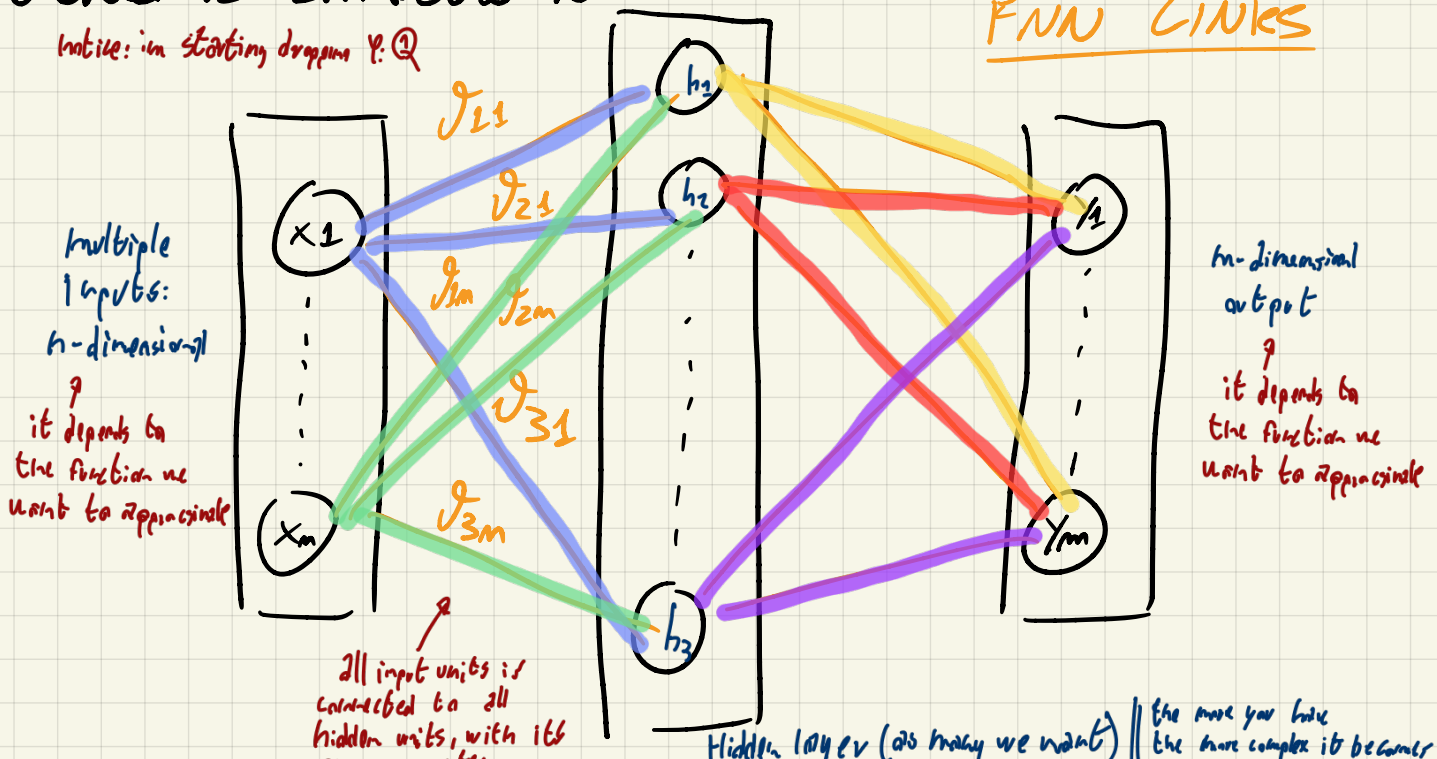
Shallow Networks are organized in layers, like this, with 1 only middle layer

|| we can add as many hidden units as we want, and it increases the accuracy

An input and an output layer could have multiple units.
In a SHALLOW FNN, there is JUST ONE HIDDEN LAYER.
Let's generalize it. $f: \mathbb{R}^n \to \mathbb{R}^m$ || Notice: from now on, you can't visualize anymore you can imagine only tridimensional stuff

GENERIC SHALLOW NN

Notice: in starting drawing ϑ: ①



In SHALLOW FNN, every unit in a layer has a link to every unit of the next layer in the model.
The function will be:

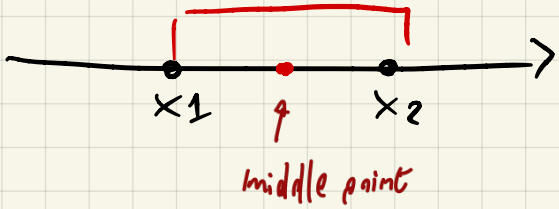
$$f(x_1, \dots x_m; \vartheta) = \langle y_1, \dots, y_m \rangle$$

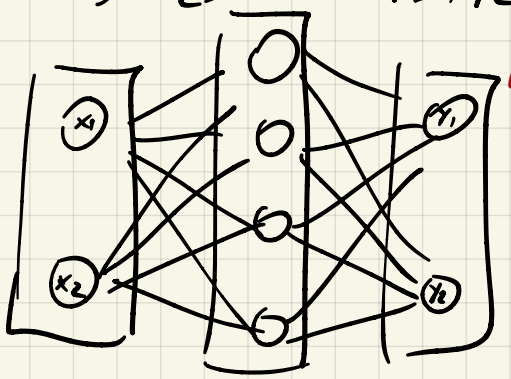
let's see an example with a function that, given two points, it gives middle distance between them on a line and their total distance:

$$\mathbb{R}^2 \longrightarrow \mathbb{R}^2$$

$$f(x_1, x_2) = \langle y_1, y_2 \rangle$$

a function that, from x1,x2, it gives the middle point and the distance





we can approximate with this

So, how powerful this FNN are?

UNIVERSAL APPROXIMATION THEOREM

it means that the output layer returns its input

A shallow FNN with a linear output layer s.t. the hidden units are "squashing"* can approximate any Borel-measurable function (example: continuous function), up to an arbitrary error ε, provided it has a sufficient number of hidden units.

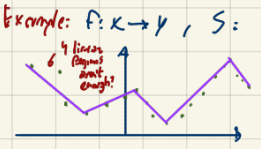
* squashing = constraining
"schiacciare" — Constrain: "limitare"

SQUASHING FUNCTION CONSTRAIN THE OUTPUT TO SOME INTERVAL:

$$\sigma: X \longrightarrow (a, b)$$

This theorem says that you can always come up with a result up to a desired error if it is used the right number of units.

example: $f: X \to Y$, S:



add one more unit

theoretically it's a very powerful model, because you can do everything with basic, but in practice not always you can do it!

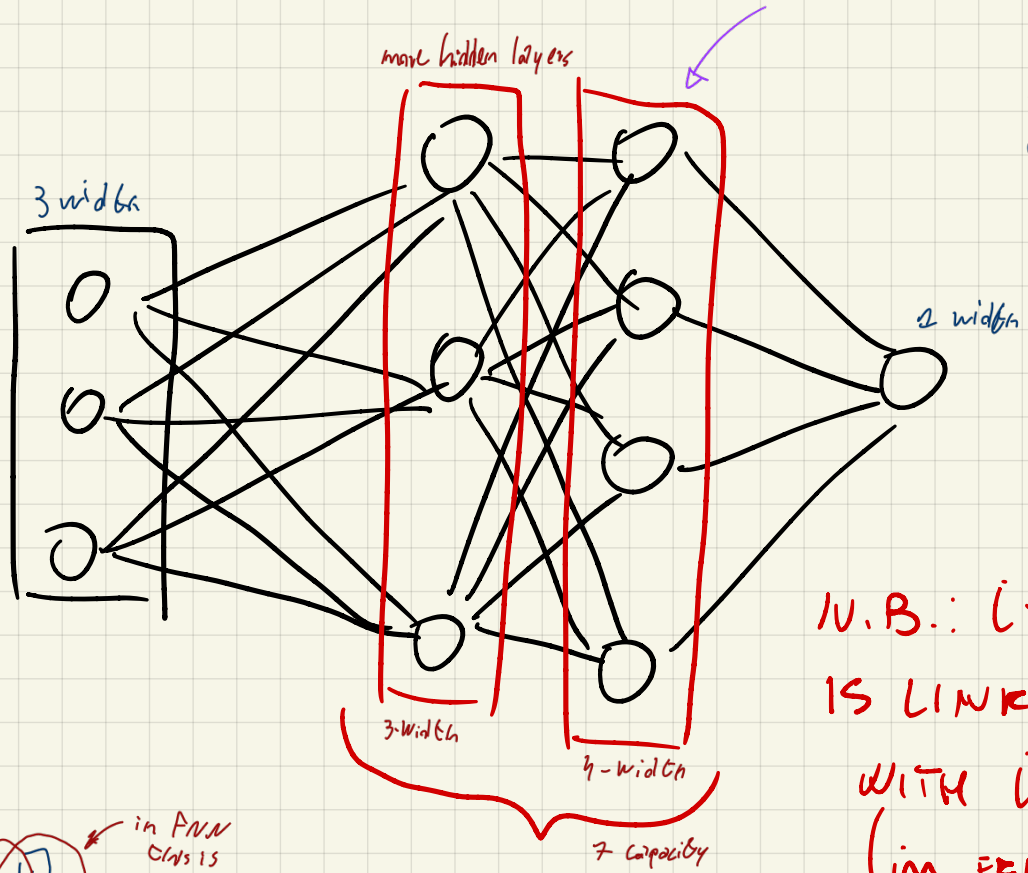
So, a SHALLOW NETWORK is built by a single hidden layer. Let's try to expand our hidden layer.

First, let's define two terms:

- CAPACITY: # of hidden units : number of hidden units
- WIDTH (of a layer): # of units in the layer (in general)

Keep in mind that DIFFERENT HIDDEN LAYERS can have DIFFERENT CAPACITY.

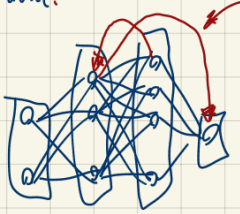
When a network has more than a single hidden layer, we talk about DEEP NETWORKS:



in general, in DEEP FNN we have [massive number of] units and HIDDEN LAYERS.
↳ usually they are measured as "parameters" and are like billions (in LLM are 200 billions)

N.B.: i-th LAYER IS LINKED ONLY WITH i+1-th LAYER (in FEED-FORWARD NN)

Notice:



in FNN this is not possible, in general neural networks yes

> 1 HIDDEN LAYERS

↓

DEEP

Notice: In ∃ already a Full course of Deep Learning, this is just an introduction of Deep Learning

We know that a SHALLOW NN can approximate any function, so why do we need a DEEP NETWORK?

First, DEEP NETWORKS can have more linear regions. → by a lot! (more efficient)

Also, for some mysterious reasons, they are faster to train and generalize better on unseen examples.

For these reasons, they are preferred with respect to Shallow FNN. However, they are harder to understand

So, deep networks are relatively better, but are very hard to understand and to use.

NEURAL NETWORK STRUCTURE

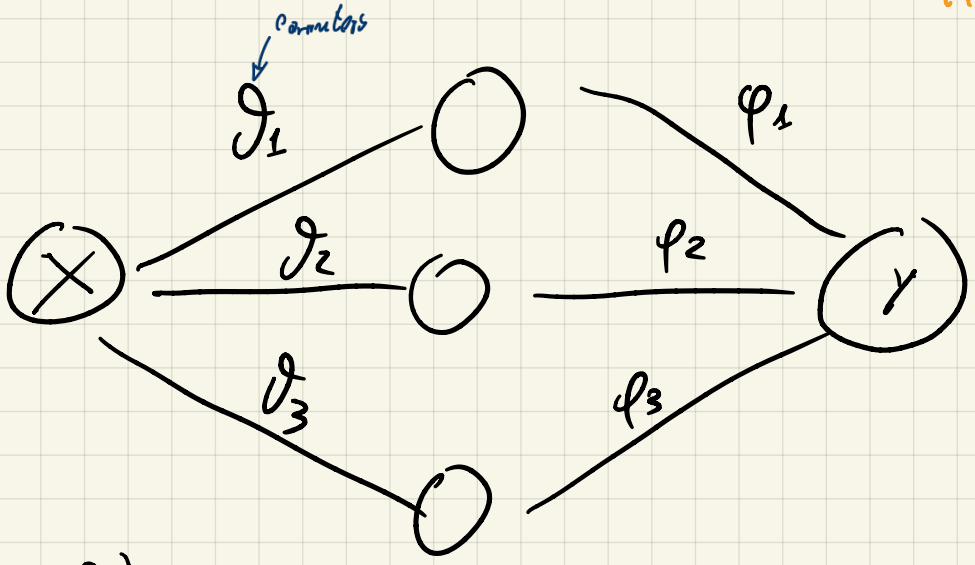
- WIDTH
- DEPTH

THERE ARE NO GUIDELINES ON HOW MUCH WIDTH AND DEPTH YOU NEED
So we have to test, and see how it performs

- ACTIVATION FUNCTIONS (HIDDEN LAYER, OUTPUT LAYER)
- LOSS FUNCTION

THESE ARE DEEPLY PROBLEM-RELATED

EXAMPLE



$$y = h(x; \vartheta)$$

In the models we have seen so far, for a given x, y is an approximation (or prediction) of $f(x)$:

$$y = \underset{\text{Prediction}}{h(x)} \sim f(x), \quad f: \mathbb{R} \to \mathbb{R}$$

the same happens for classification, but with probability:

$$f: \mathbb{R}^3 \to \{0, 1\}$$

$$y = h((x_1, x_2, x_3)) \sim f(x) \in \{0, 1\}$$

$$h((x_1, x_2, x_3)) = \text{probability} \quad \| \text{for example, the probability that this is Class 1}$$

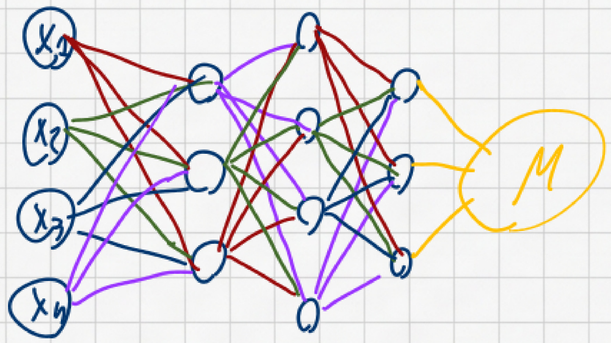
So we are saying: Not only we can use FNN to construct a proper predictor, but we can also decide to use it for Classification!

Instead of constructing a FNN such that, given a input x, produce the simple probability value (what we do in general for Binary Classification) we decide to produce the parameters of the Conditional Probability Distribution (s.t. σ on μ for the Gaussian Distribution).

FNN as an Gaussin Output Predictor:

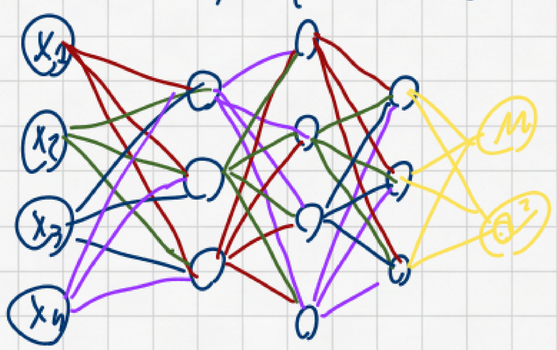
If we have to predict a continous value (like temperatures or houses prices), it is typically assumed that those outputs follows a Gaussian Distribution. In this case, we can configure the Neural Network in a way that the last layer produces the required parameters to draw the Gaussian function:

- Omoscedastic Regression (only the μ mean): In the most common scienario, it is used a pure linear combination in a way to produce only the μ mean of the Gaussian Distribution:



the σ^2 variance is ignored for optimizations or treated as a fixed value, which mathematically brings to a Squared Mean Error as a Loss function

- Heteroscedatic Regression (mean μ an variance σ^2): To be more precise, it is possible to produce both μ and σ^2, which respects to the input x. For doing it, the output σ^2 in general will be squared s.t. it will be always a positive value.



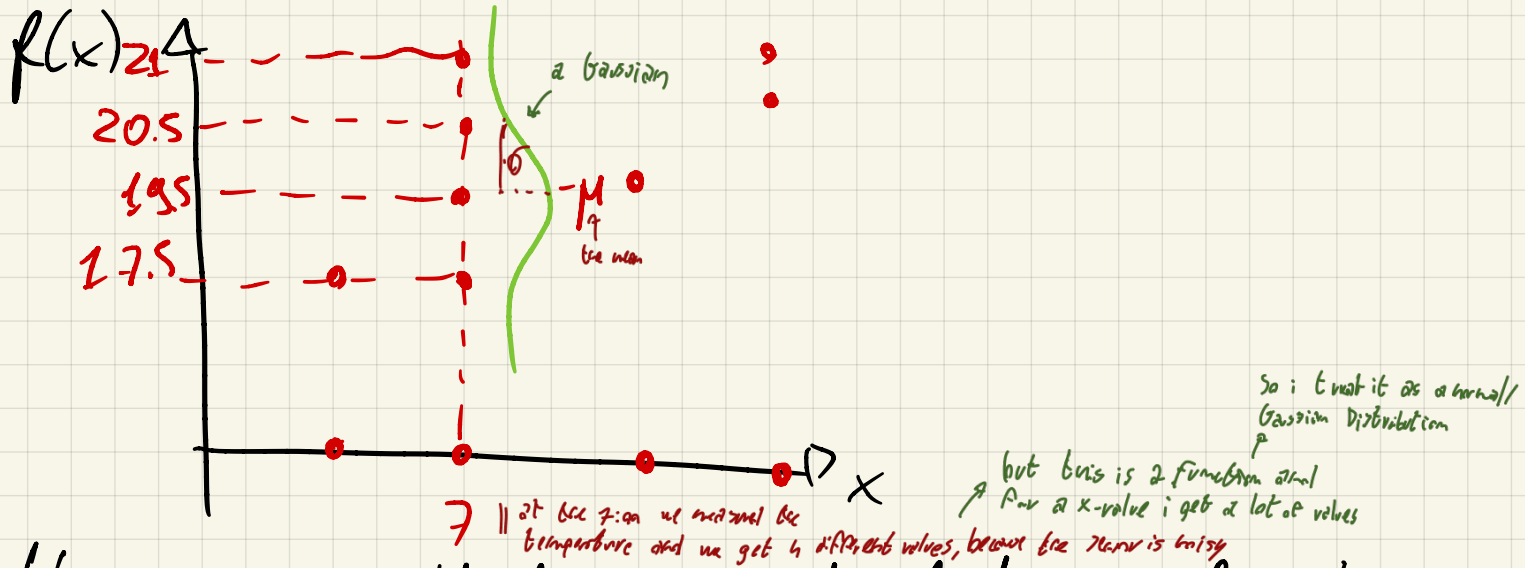
FNN as a Classifier

When you want that the result is simply 0 or 1, we consider Bernulli's Distribution (for binary distributions) or Categoric Distribution (for Multi-Class Classification).

"FNN as a Gaussian Predictor"

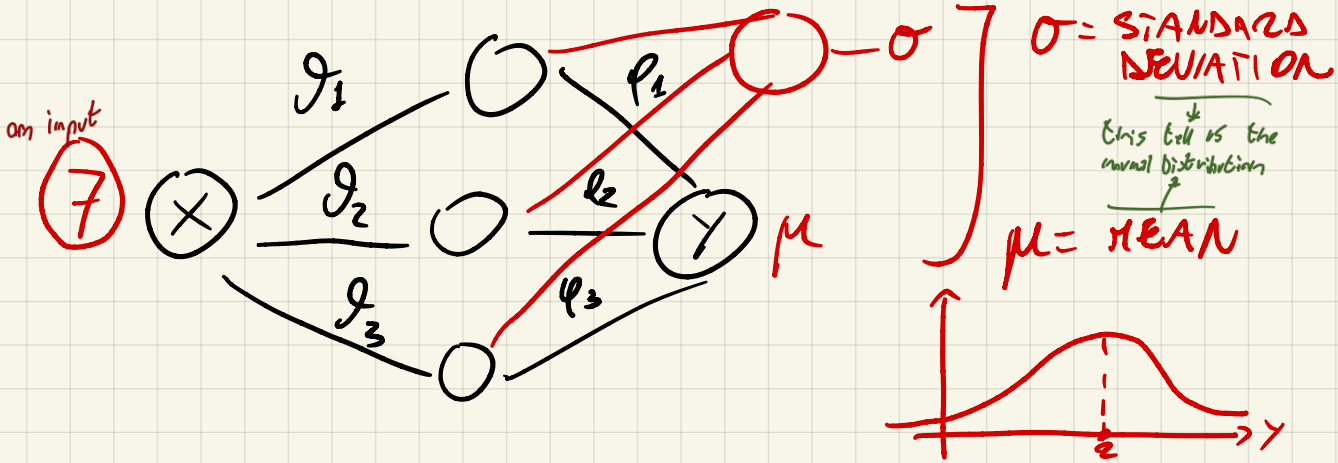
With neural networks, for a given input, the output will be a probability distribution of the possible output.

For example, let's see a very noisy dataset, where for a given x in a specific time the temperature measurement has been measured different for 10 times:



Let's imagine that our function retrieves a distribution of the temperature values. So I'm just computing the mean of a Gaussian distribution.

On a neural network, although, we retrieve the probability distribution of those values.
In example, let's assume our network has given a conditional probability distribution as output.



So, our network will retrieve a mean with a standard deviation from the mean.

FNN as a Classifier

So here the difference is that instead of studying the most probable output from a distribution, here I am retrieving the probability distribution as output of the network.

In example, for a binary classification, we are retrieving the probability distribution of the classification output between the whole dataset.

$$p(t|\lambda) = \begin{cases} 1-\lambda, & y=0 \\ \lambda, & y=1 \end{cases}$$

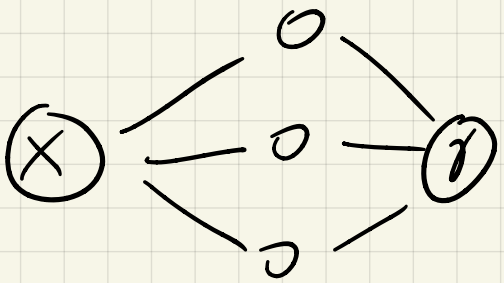
λ: parameter of the probability distribution

It is a BERNOULLI DISTRIBUTION that could be written as:

$$p(t|\lambda) = (1-\lambda)^{1-t} \cdot \lambda^{t}$$

BERNOULLI DISTRIBUTION

The output parameter of the network, y, will be interpreted as λ:



$$y(7) = 0.62 = \lambda$$

$$p(1|y(7)) = (1-0.62)^{0} \cdot 0.62^{1} = 0.62$$

Notice: that's a probability, the activation function (sigmoid) should return 1 here

$$p(0|y(7)) = (1-0.62)^{1} \cdot 0.62^{0} = 1-0.62$$

So, the network predicted the probability of x to be 1. Then, I can analyze which is the probability of this prediction using a conditional distribution.

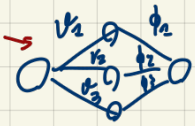
Back to our problem, my goal is to maximize the fitting of the prediction/classification. So for a given training set S:

$$S = \{(x_1, t_1), (x_2, t_2), \dots, (x_m, t_m)\}$$

(value, tag)

I want to maximize the probability of x_i being t_i. (we still assume that the distribution is uniform):

$$p(t \mid x, \vartheta)$$

→ I want to find the parameters →  → such that the error in minimum

$$\hat{\vartheta} = \underset{\vartheta}{\operatorname{argmax}} \prod_{i=1}^{m} p(t_i \mid x_i; \vartheta) = \underset{\vartheta}{\operatorname{argmax}} \prod_{(x,t)\in S} p(t \mid x; \vartheta)$$

$\hat{\vartheta}$: the perfect parameters such that maximize the probability such that $\forall x_i$ you get the corresponding t_i

MAXIMUM LIKELIHOOD PRINCIPLE

can be extremely tiny because we are multiply probabilities

N.B.: our ϑ can be tiny due to APPROXIMATION ERROR (for a large scale of parameters)

Also, $\hat{\vartheta}$ can be explicited this way (to find log likelihood):

$$\hat{\vartheta} = \underset{\vartheta}{\operatorname{argmax}} \log\left(\prod_{(x,t)\in S} (p(t \mid x; \vartheta))\right) =$$

We decide to use the logarithm for the reason: $\left(\log\left(\prod_i a_i\right) = \sum_i \log(a_i)\right)$

$$= \underset{\vartheta}{\operatorname{argmin}} -\log\left(\prod_{(x,t)\in S} (p(t \mid x; \vartheta))\right) =$$

In this way, computers can calculate with high precision without gaining in "underflow" without changing the original objective because the logarithm function is monotone

$$= \underset{\vartheta}{\operatorname{argmin}} \underbrace{\sum_{(x,t)\in S} -\log\left(p(t \mid x; \vartheta)\right)}_{\text{LOSS FUNCTION } J(\vartheta)} = \underset{\vartheta}{\operatorname{argmin}}\, J(\vartheta)$$

(NEGATIVE LOG LIKELIHOOD)

This is the loss function because it is what we want to minimize to fit the data.

So Now We Have All The Recipe

I tuoi quattro punti riassumono perfettamente i passaggi formali per la progettazione e l'uso di una rete neurale [1] [...]. Ecco come le tue intuizioni si mappano passo dopo passo sulla teoria matematica:

1) Capisci che distribuzione parametrica ci interessa Questo è il primissimo passo: guardi il tipo di dati che vuoi prevedere (il dominio di output) e scegli la distribuzione di probabilità che meglio li descrive [2].

- Se prevedi numeri continui (regressione), usi una distribuzione **Normale (Gaussiana)** [4] [5].
- Se classifichi tra due sole opzioni (vero/falso), usi una distribuzione di **Bernoulli** [6].
- Se classifichi tra K opzioni multiple (es. riconoscere cifre), usi una distribuzione **Categorica** [7].

2) Capisci che parametri prevedere di tale distribuzione Una volta scelta la distribuzione, decidi quali parametri la rete neurale dovrà generare [2] [4]. Costruisci l'ultimo strato (output layer) della tua rete affinché rispetti i vincoli matematici di quei parametri:

- Se la rete deve prevedere la probabilità λ della Bernoulli (che deve stare tra 0 e 1), userai come attivazione finale una **Sigmoide** [8].
- Se devi prevedere le probabilità K della Categorica (che devono sommare a 1), userai una **Softmax** [9].
- Se devi prevedere la media μ della Gaussiana (che può essere qualsiasi numero), lascerai un **output lineare** [5].

3) Trova i parametri ottimali (Fase di Training) Adesso che il modello è impostato per generare distribuzioni, devi addestrarlo [3]. L'obiettivo è trovare l'insieme dei parametri ottimali della rete neurale (i pesi e i bias) in modo da rendere il più probabile possibile l'osservazione dei dati del tuo training set [10]. Lo fai iterativamente minimizzando la tua Funzione di Perdita (Loss Function), che altro non è se non la **Negative Log-Likelihood** della distribuzione che hai scelto al punto 1 [3] [...].

4) Testa ed usa il modello (Fase di Inference) Il modello ora è pronto e ottimizzato. Quando gli passi un nuovo input x mai visto prima, la rete calcola i parametri della distribuzione di probabilità per quell'input [3] [13]. Per prendere la tua decisione finale e ottenere una singola previsione (point estimate), vai semplicemente a estrarre il valore di output che massimizza quella probabilità calcolata dalla rete (operazione di *argmax*) [1] [13]. Ad esempio, per la normale restituirai direttamente la media μ prevista, essendo il picco della campana [14].

Questo schema a 4 step è incredibilmente potente perché ti permette di affrontare un'infinità di task diversi (regressione, classificazione, conteggio di eventi, ecc.) semplicemente cambiando la distribuzione di partenza, senza dover reinventare la matematica dell'apprendimento da zero [15] [16].

Let's now see the general approach to design the loss function, in steps:

1. FIX PARAMETRIZED DISTRIBUTION OVER PREDICTIONS

$$p(t|\lambda)$$

2. SET THE MODEL (THE NN) TO PREDICT (SOME) PARAMETERS OF THE DISTRIBUTION

Not the output!

↳ because depends if we are actually using all the parameters

$h(x;\vartheta)$ NETWORK OUTPUT

if $h(x;\vartheta)$ predicts $p(t|\lambda)$, then:

$$p(t|\lambda) = p(t|x;\vartheta)$$

3. TRAIN THE MODEL TO MINIMIZE $J(\vartheta)$ OVER THE TRAINING SET

$$\hat{\vartheta} = \arg\min_{\vartheta} J(\vartheta)$$

4. GIVEN (SOME) x, PREDICT THE OUTPUT BASED ON:

$y(x) = \arg\max_{t} p(t|x;\hat{\vartheta})$ VALUE THAT MAXIMIZES t

---

Let's now see for every type of optimization model our NN work:

We are applying the recipe we saw above for each kind of usage

REGRESSION

$f: \mathbb{R}^n \to \mathbb{R}$

1. FIX DISTRIBUTION: $p(t|\mu,\sigma)$

Do we really need σ? No

When i compute the negative log likelihood i find: $-\log\left(\frac{a}{\sqrt{2\pi\sigma^2}}\right) + \frac{(y-\mu)^2}{2\sigma^2}$

we have to find weights and bias such that minimize this value, but as we can see, the first value term is constant, so since zero value, $2\sigma^2$ is just a positive value

$= \mathcal{N}(\mu,\sigma)$

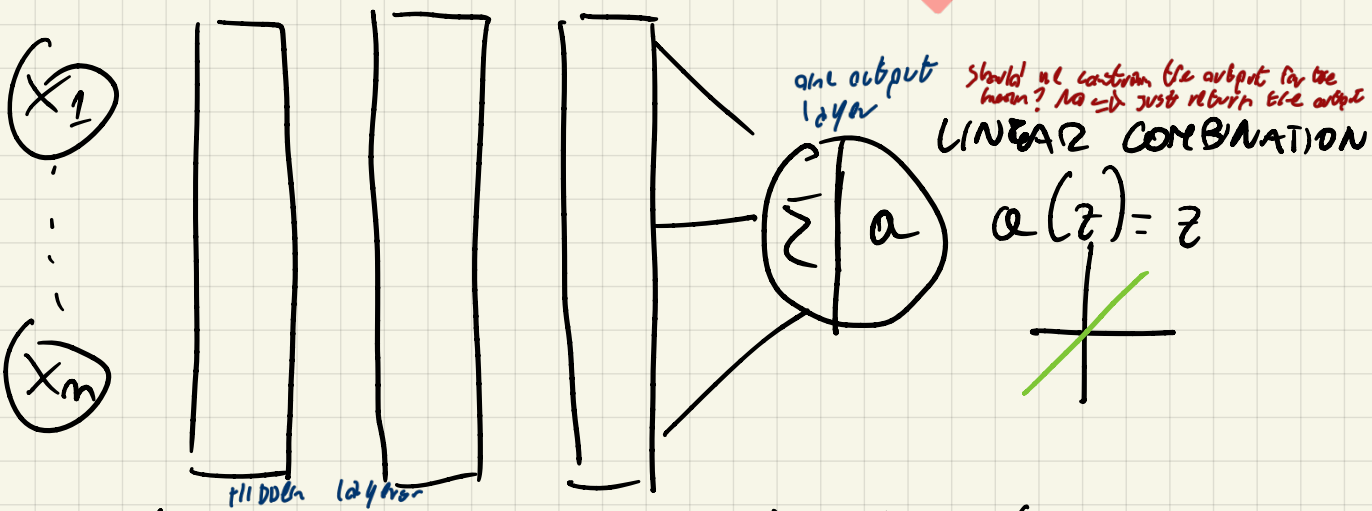
NORMAL DISTRIBUTION

$$\Sigma = \begin{bmatrix} \sigma_1 & & \emptyset \\ & \sigma_2 & \\ \emptyset & & \sigma_m \end{bmatrix}$$

$p(t|\mu,\Sigma)$ IN A MULTIVARIANT CASE

2. DESIGN NETWORK TO PREDICT DISTRIBUTION PARAMETER(S).

In this case, we predict just the mean (μ). ← only the mean



$$\mu = h(\vec{x}; \vartheta) \quad ; \quad p(t|\mu, \sigma) = p(t|h(\vec{x}, \vartheta), \sigma) = \mathcal{N}(h(\vec{x}; \vartheta), \sigma)$$

$h(\vec{x}; \vartheta)$: what the output computes given the parameters ϑ

σ: is unknown, do we need it? No

ϑ is unknown, but we don't need it for a maximization problem

3. MODEL TRAINING (What is the loss function?)

$$J(\vartheta) = \sum_{(x,t)\in S} -\log \mathcal{N}(t|h(\vec{x}; \vartheta), \sigma) \propto \sum -\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) + \frac{(t - h(x,\vartheta))^2}{2\sigma^2}$$

not important for argmin $J(\vartheta)$

Notice:
This is exactly what we did with linear regression. Indeed:

$$J'(\vartheta) = \sum_{(x,t)\in S} \|h(x;\vartheta) - t\|^2$$

(*) because of the fact that it isn't important i can consider another $J'(\vartheta)$ function that consider only the $(t - h(x,\vartheta))^2$ value.

‖ squared mean error → this is typically what we use to do regression with NN

$$\operatorname{argmin}_\vartheta J(\vartheta) = \operatorname{argmin}_\vartheta J'(\vartheta)$$

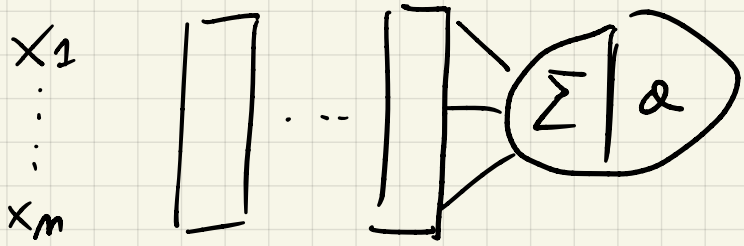
PROOF ON THE BOOK

What changes if I have, for n inputs, m outputs? I will just have a distribution over a vector ($\mu \to \vec{\mu}$, $\sigma \to \vec{\sigma}$), where every unit is still linear (i.e., every unit has $a(z) = z$). While training, our expected t will be a vector of expected values and:

$$J'(\vartheta) = \sum\sum \|h(x;\vartheta) - t\|^2$$

BINARY CLASSIFICATION

1) $p(t|\lambda) = (1-\lambda)^{1-t} \cdot \lambda^{t}$ | Bernoulli distribution

2) 

$a(\vartheta) = sig(\vartheta) = \frac{e^{\vartheta}}{1+e^{-\vartheta}} = \sigma(\vartheta) \in \{0,1\}$

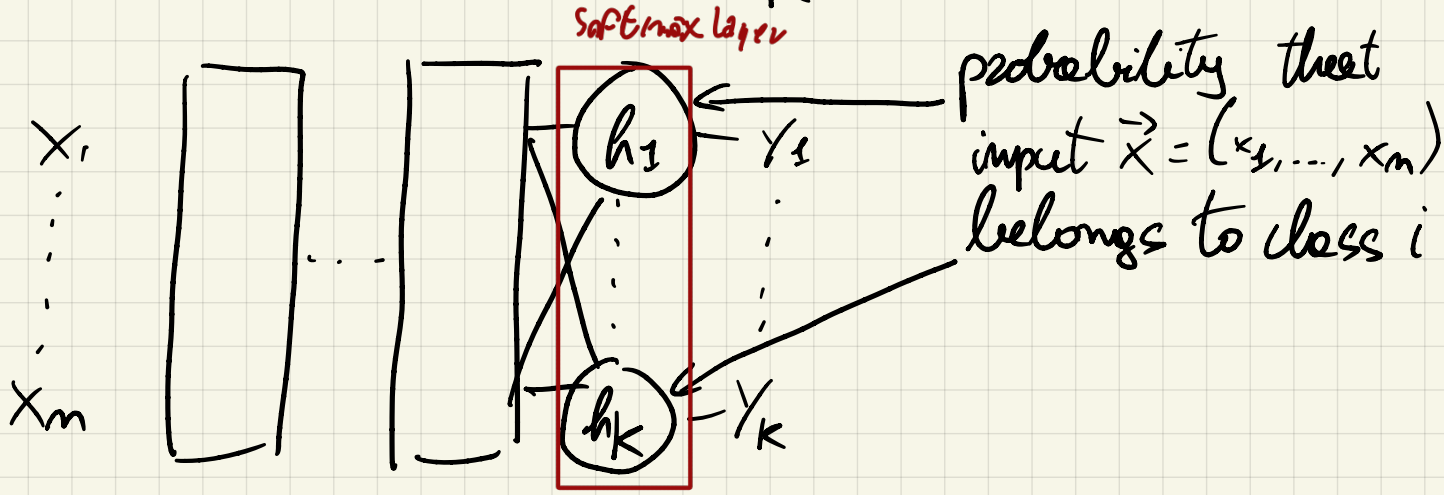
$$p(t|h(x;\vartheta)) = p(t|x) = (1-\sigma(h(x);\vartheta))^{1-t} \cdot \sigma(h(x);\vartheta)^{t}$$

3) $$J(\vartheta) = -\log \prod_{(x,t)\in S} (p(t|x)) = \sum_{(x,t)\in S} -(1-t)\cdot \log(1-\sigma(h(x);\vartheta)) - t\cdot \log(\sigma(h(x);\vartheta))$$

BINARY CROSS-ENTROPY

MULTI-CLASS CLASSIFICATION

Here, we will see also the network architecture build.
Let's have k classes: K-classes



λ_i: probability that input belongs to class i

$$p(y=k) = \lambda_k$$

To fix the outputs, we will pass them to a softmax layer:

$$y_i = softmax_i(h) = \frac{e^{h_i}}{\sum_{s=1}^{K} e^{h_s}}$$

$$p(y=k|x) = \text{softmax}_k(h(x;\vartheta))$$

$$J(\vartheta) = -\sum_{(x,t)\in S} \log\left(\text{softmax}_t(h(x;\vartheta))\right) = \quad \text{MULTI-CLASS CROSS-ENTROPY}$$

$$= -\sum_{(x,t)\in S} h_t(x;\vartheta) - \log \sum_{j=1}^{k} e^{h_j(x;\vartheta)}$$

Using exponentials for softmax normalization AMPLIFIES THE DIFFERENCES between all classes (you could use also other operations/functions)

N.B.: each output layer here DEPENDS ON EACH OTHER

To resume:

1. REGRESSION
   - OUTPUT: LINEAR ← Basically, there is no activation function
   - LOSS: SUM OF SQUARED ERRORS
2. BINARY CLASSIFICATION
   - OUTPUT: SIGMOID → IT'S A SQUASHING FUNCTION!
   - LOSS: BINARY CROSS-ENTROPY
3. MULTICLASS CLASSIFICATION
   - OUTPUT: SOFTMAX
   - LOSS: MULTICLASS (CATEGORICAL) CROSS-ENTROPY

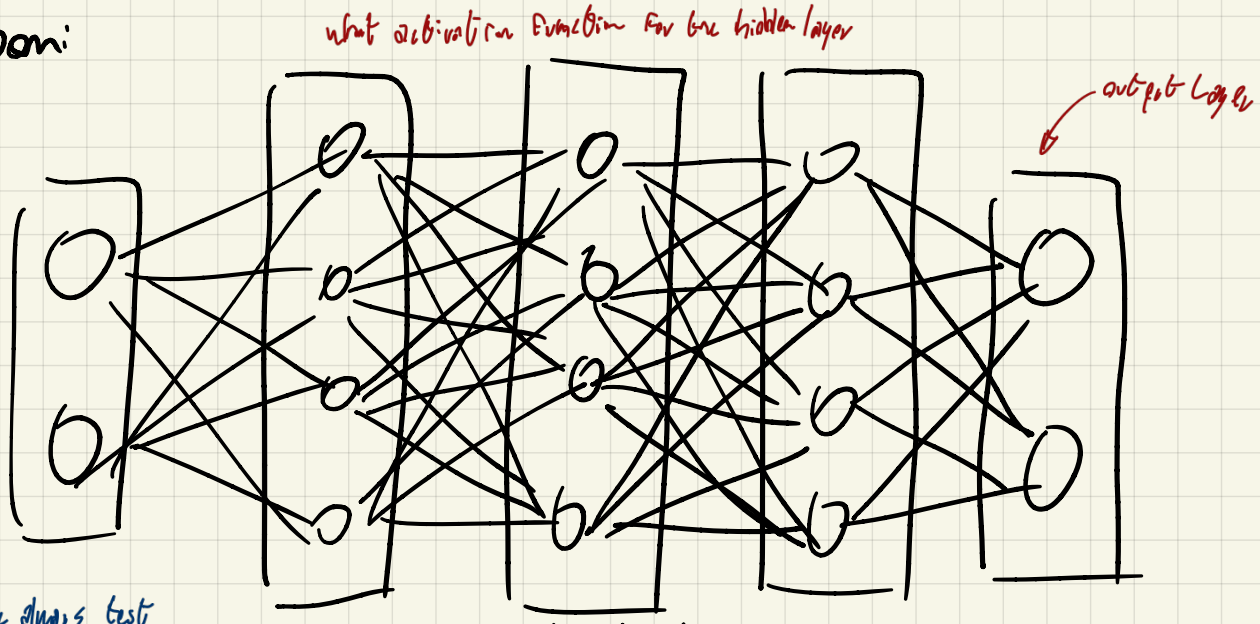
What remains to discuss are the ACTIVATION FUNCTIONS for hidden units.

REMINDER: we are using FULLY CONNECTED NEURAL NETWORKS

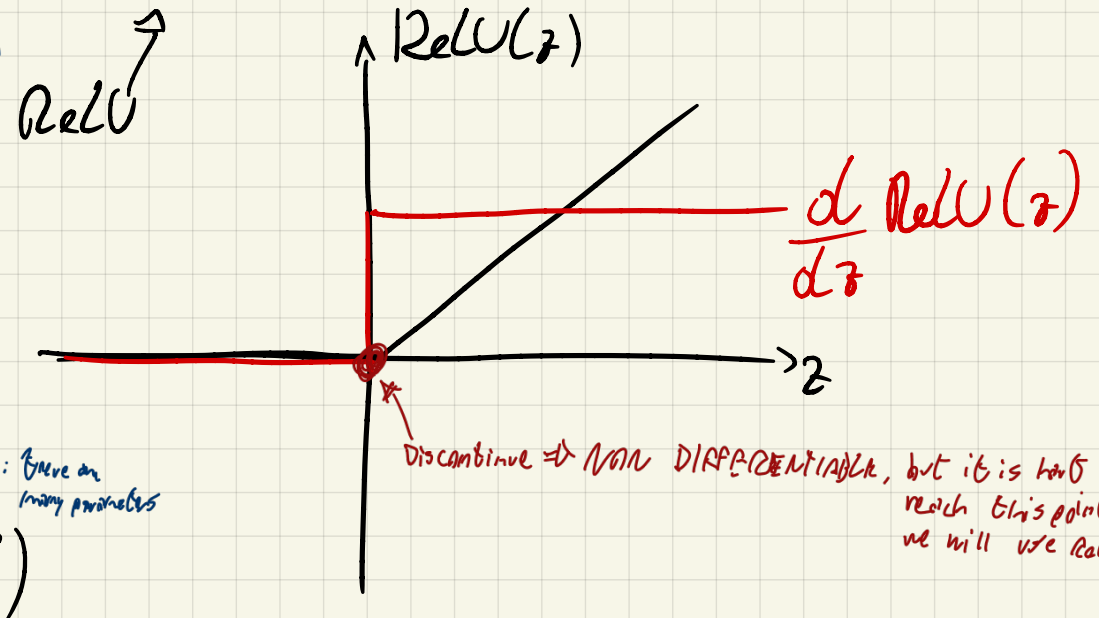
Basically, activation functions depend by the specific architecture we are dealing with.

Notice! We saw the Gradient Descent, but in nonlinear functions you can no longer find the minimum loss function so you simply do your best → we can it a LOCAL ECONOMY

let's see an example while applying ReLU(x) as an activation function:



In general, you have always test all possible activation functions for the hidden layer, but the most common one is the RELU function



Remember: there are many parameters

$$y = f(\vec{x}; \vec{\vartheta})$$

otherwise we cannot use the GRADIENT DESCENT

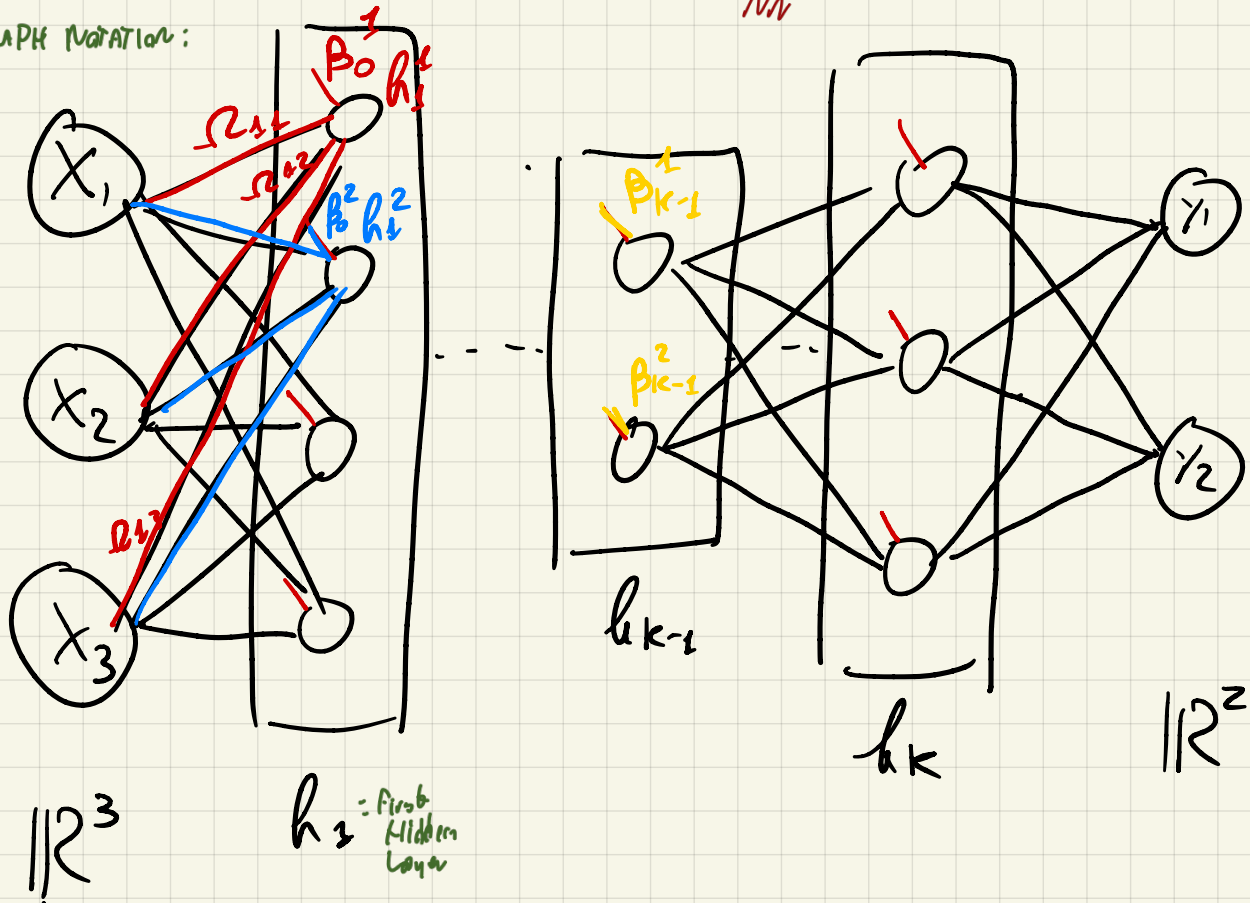
N.B.: ACTIVATION FUNCTIONS MUST BE DIFFERENTIABLE

So, in this case we will just apply GRADIENT DESCENT in order to minimize the loss function.

In general, we used graphs to represent NN. let's see a more convenient representation:

MATRIX NOTATION FOR NN → it allows us to compactly represent NN

GRAPH NOTATION:



α: activation function for hidden layers

$$h_1^1 = \alpha\left[\Omega_{11}x_1 + \Omega_{12}x_2 + \Omega_{13}x_3 + \beta_0^1\right] = \alpha\left(\begin{bmatrix}\Omega_{11} & \Omega_{12} & \Omega_{13}\end{bmatrix}\begin{bmatrix}x_1\\x_2\\x_3\end{bmatrix} + \beta_0^1\right)$$

h_1^1: 1 = First Unit of the First Layer; β_0^1: Bias

$$h_1^2 = \alpha\left[\Omega_{21}x_1 + \Omega_{22}x_2 + \Omega_{23}x_3 + \beta_0^2\right] = \alpha\left(\begin{bmatrix}\Omega_{21} & \Omega_{22} & \Omega_{23}\end{bmatrix}\begin{bmatrix}x_1\\x_2\\x_3\end{bmatrix} + \beta_0^2\right)$$

I can put together h_1^1 and h_1^2:

$$\begin{bmatrix} h_1^1 \\ h_1^2 \end{bmatrix} = a\left(\begin{bmatrix} \Omega_{11} & \Omega_{12} & \Omega_{13} \\ \Omega_{21} & \Omega_{22} & \Omega_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} \beta_0^1 \\ \beta_0^2 \end{bmatrix} \right) = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

ELEMENT-WISE APPLICATION (YOU APPLY a TO EVERY ELEMENT)

So we can generalize the notation as:

$$\underbrace{\begin{bmatrix} h_1^1 \\ h_1^2 \\ h_1^3 \\ h_1^4 \end{bmatrix}}_{h_1} = a\left(\underbrace{\begin{bmatrix} \Omega_{11} & \Omega_{12} & \Omega_{13} \\ \vdots & & \vdots \\ \Omega_{41} & \cdots & \Omega_{43} \end{bmatrix}}_{\Omega_0} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{\times} + \underbrace{\begin{bmatrix} \beta_0^1 \\ \vdots \\ \beta_0^4 \end{bmatrix}}_{\beta_0} \right)$$

$$h_1 = a(\Omega_0 \vec{x} + \vec{\beta_0}) = a(\Omega_0 x + \beta_0)$$

So, for the other layers:

$$h_2 = a(\Omega_1 h_1 + \beta_1)$$

$$\vdots$$

$$h_{k-1} = a(\Omega_{k-2} h_{k-2} + \beta_{k-2})$$

$$h_k = a(\Omega_{k-1} h_{k-1} + \beta_{k-1})$$

$$y = \underbrace{o}_{\text{notice } o \neq a}(\Omega_k h_k + \beta_k) \Rightarrow$$

OUTPUT ACTIVATION FUNCTION

↳ Depends by the problem: Binary Classification → Sigmoid, Multiclass Classification → Softmax, Summ, Softmax

Notice: the activation function can be different for each hidden layer but in general is ELU

The OUTPUT ACTIVATION FUNCTION is the act. function applied in the output units. Based on which output we want, that function could be a sigmoid, a softmax, ecc as we discussed before.

TRAINING AN MODEL (FITTING)

Our objective is minimizing the loss function. Let's see it for the case of linear regression:

likelihood: the probability that the predicting produced by our Network, matches the actual correct value

$$\hat{\varphi} = \underset{\varphi}{\operatorname{argmin}} \left[L(\varphi) \right]$$

output

$$f(x;\varphi) = \varphi_0 + \varphi_1 x_1 + \varphi_2 x_2 + \varphi_3 x_3$$

$$L(\varphi) = \sum_{(x,y)\in S} \left(f(x;\varphi) - y \right)^2$$

OUR SET IS FIXED, SO WE ONLY NEED TO CHANGE THE PARAMETERS S.T. $L(\varphi)$ IS MINIMIZED

the Gradient

$$\frac{\partial L}{\partial \varphi} = \begin{bmatrix} \frac{\partial L}{\partial \varphi_0} \\ \frac{\partial L}{\partial \varphi_1} \\ \vdots \\ \frac{\partial L}{\partial \varphi_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial \varphi_0}(\bar{\varphi}) \\ \vdots \\ \frac{\partial L}{\partial \varphi_3}(\bar{\varphi}) \end{bmatrix}$$

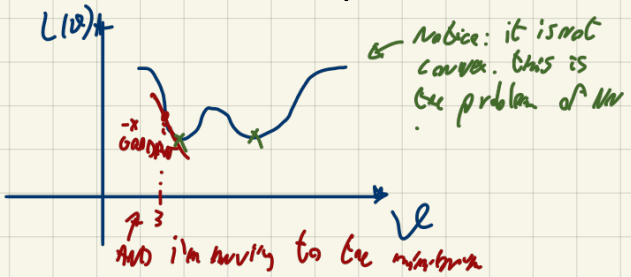
N.B.: THIS FORMULA IS WRITTEN FOR RANDOM $\varphi_0, \varphi_1, \varphi_2, \varphi_3$ VALUES.

OF COURSE, FOR A GIVEN PARAMETER WE USE THE CURRENT VALUE OF THE PARAMETER $\left(\bar{\varphi} = (\bar{\varphi}_0, \bar{\varphi}_1, \bar{\varphi}_2, \bar{\varphi}_3)\right)$

So, after computing gradient descent, I update my φ:

$$\varphi \longleftarrow \bar{\varphi} - \alpha \frac{\partial L}{\partial \varphi}(\bar{\varphi})$$

α = LEARNING RATE



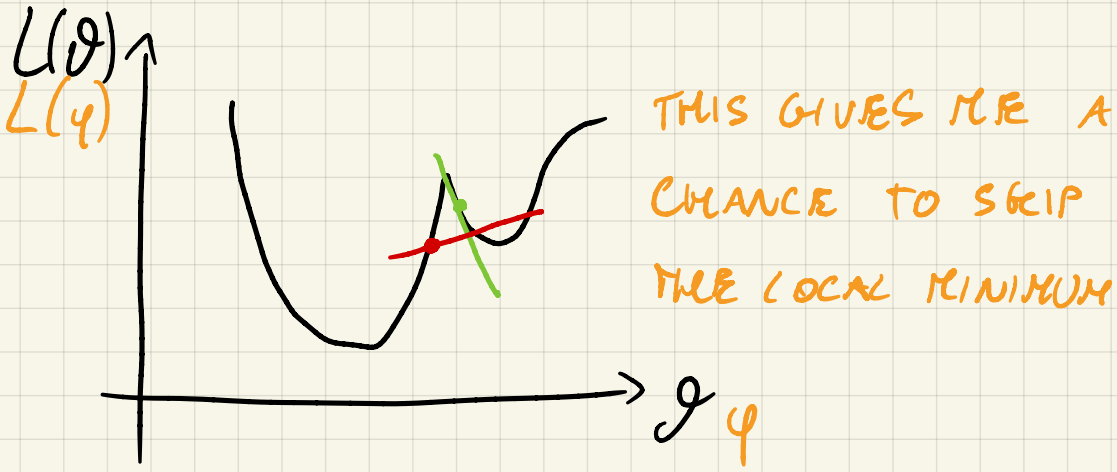
- Now, if I proceed I could get trapped on a local minimum, but this just depends on how I started from.
- The second problem is that derivating partially is not so easy, because it is needed to apply the gradient descent w.r.t. all the parameters.

Given the NN of the last chapter, if i.e. my S is:

$$S = \{(3,2)\} \rightarrow L((y_1, y_2), (x_1, x_2))$$

To plot our L into S, I have to compute $\frac{\partial L}{\partial \varphi}$, but we have to do it for the composing functions $\frac{\partial}{\partial \varphi}$ of all the hidden units that are computing a derivative w.r.t. our parameters.

We try to partially solve the problem by using a slightly different gradient descent by adding SOME NOISE (which is RANDOM):



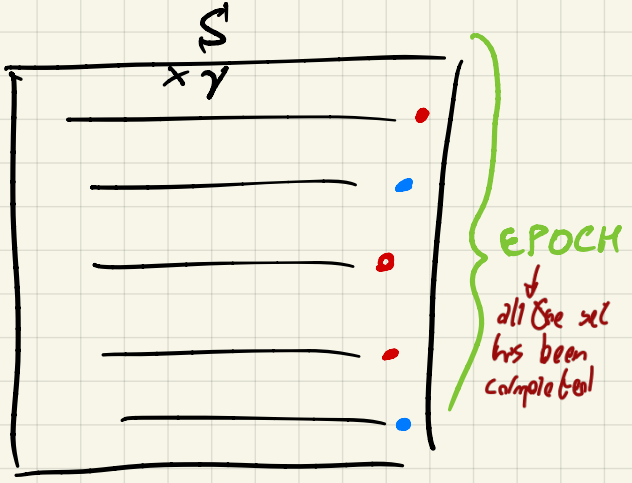
Instead of computing the loss function by using the whole S:

$$L(\varphi) = \sum_{(x,y) \in S} |f(x;\varphi) - y|^2$$

I apply STOCHASTIC GRADIENT DESCENT by randomly mini-batching.

B = mini-batch of S:

$$L(\varphi) = \sum_{(x,y)\in B} |f(x;\varphi) - y|^2$$



When the iteration is over, a new mini-batch is randomly picked w/out the elements of the previous one(s). While the full S has being iterated, we say over an EPOCH. An algorithm can pass through more epochs, as it increases the possibility to find the global minimum.

MOMENTUM (modify the stochastic gradient function, because the SGD can be too much oscillant)

At the next iteration, I compute the gradient this way:

$$m_{t+1} \longleftarrow \beta \cdot m_t + (1-\beta) \cdot \sum_B \frac{\partial L(\varphi_t)}{\partial \varphi}$$

MOMENTUM

$$\varphi_{t+1} \longleftarrow \varphi_t - \alpha m_{t+1}$$

φ_t : state of parameters at iteration t

$\beta \in [0, 1)$

If $\beta = 0$, m_{t+1} is just our gradient descent. If it's not, I am combining the gradient I have previously computed with the more previous ones, where the greater is β, the more importance the previous ones have for $t+1$. This is done batch by batch.

ADAM || That's the standard training approach, we want to make learning stable and efficient

Using gradient descent brings a problem of stability based on the function: the steeper is my function, the longer will be the time to reach the minimum, and viceversa. Empirically it is tested that the loss function is explodable properly with ADAM (theoretical proof is poor though). ADAM is based on making fixed steps on the function:

$$m_{t+1} \leftarrow \frac{\partial L(\varphi_t)}{\partial \varphi}$$

← we compute the gradient

$$v_{t+1} \leftarrow \left(\frac{\partial L(\varphi_t)}{\partial \varphi}\right)^2$$

we also the squared gradient

we are squaring because it may be negative then it will be squared

Let's fix the steps according to this formula:

NORMALIZED GRADIENT

$$\varphi_{t+1} \leftarrow \varphi_t - 2 \frac{m_{t+1}}{\sqrt{v_{t+1}} + \varepsilon}$$

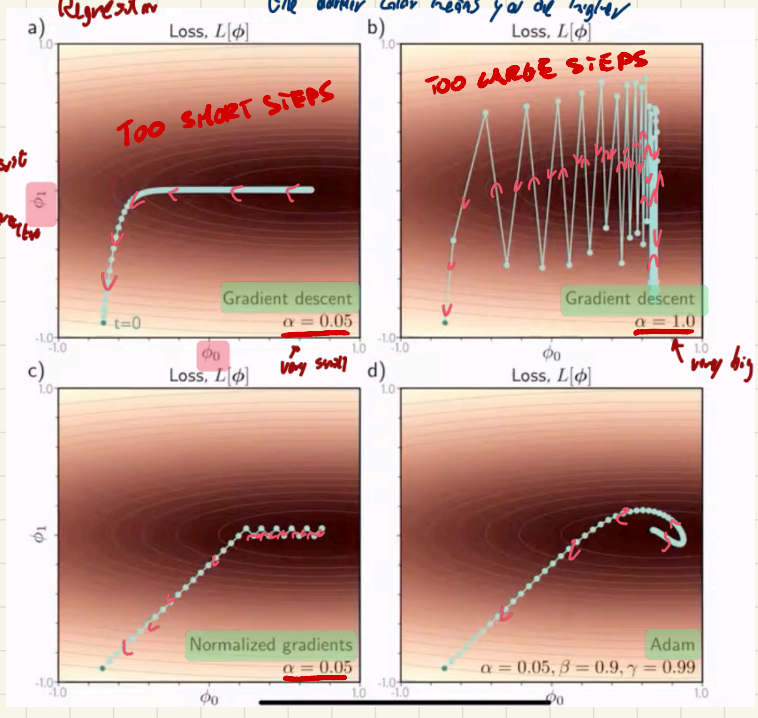
2 = learning rate

NORM = 1

the gradient might be 0, so I add a small quantity

compared to gradient descent, I am just isolating the sign of the gradient descent (that's why we use $\sqrt{v_{t+1}}$). So I am just using gradient descent direction and, by learning rate, how further my step should be done.

$$\varphi_{t+1} \leftarrow \varphi_t - 2 \frac{\partial L(\varphi_t)}{\partial \varphi}$$



LINEAR REGRESSION EXAMPLE (2 PARAMETERS: φ_0 - φ_1)

Now, by mixing MOMENTUM and NORMALIZED GRADIENT:

these algorithms are also called "optimizers"

ADAM

Basically I manually choose how important is the history of both gradient and squared gradient, with $\beta, \gamma \in [0,1)$

$$m_{t+1} \longleftarrow \beta m_t + (1-\beta)\frac{\partial \mathcal{L}(\varphi_t)}{\partial \varphi}, \quad \beta \in [0,1)$$

$$v_{t+1} \longleftarrow \gamma v_t + (1-\gamma)\left(\frac{\partial \mathcal{L}(\varphi_t)}{\partial \varphi}\right)^2, \quad \gamma \in [0,1)$$

$$\varphi_{t+1} \longleftarrow \varphi_t - \alpha\left(\frac{m_{t+1}}{\sqrt{v_{t+1}+\epsilon}}\right)$$

α: learning rate

→ NORM $\neq 1$

Works like SGD but with different update formulas

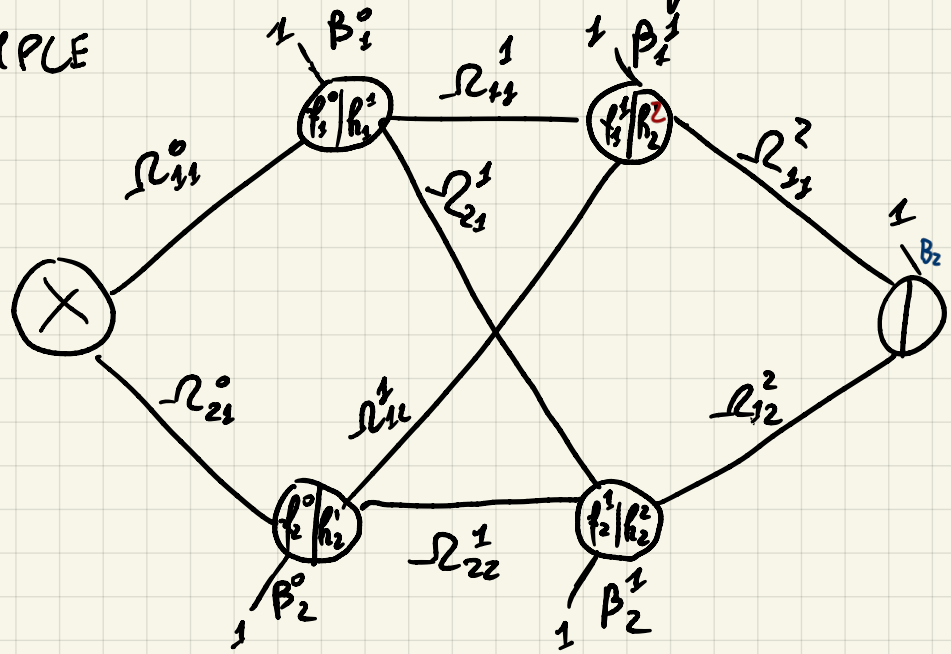
The hyperparameters of this algorithm are 3:

- learning rate (α)
- β, γ

ADAM is the general standard algorithm in order to fit a neural network to some data.

N.B.: Neural network fitting as they are seen here rely on loss function DIFFERENTIABILITY (w/out it, we can't rely to GRADIENT DESCENT, which leads to the direction where the loss function is getting reduced)

EXAMPLE



"linear combination of the input"

f: PREACTIVATION FUNCTION

h: ACTIVATION FUNCTION

$F_1^0: B_1^0 + \Omega_{11}^0 x$

$F_2^0: B_2^0 + \Omega_{21}^0 x$

$$f^0 = \begin{bmatrix} f_1^0 \\ f_2^0 \end{bmatrix} = \begin{bmatrix} \beta_1^0 + \Omega_{11}^0 x \\ \beta_2^0 + \Omega_{21}^0 x \end{bmatrix} = \underbrace{\begin{bmatrix} \beta_1^0 \\ \beta_2^0 \end{bmatrix}}_{\beta^0} + \underbrace{\begin{bmatrix} \Omega_{11}^0 \\ \Omega_{21}^0 \end{bmatrix}}_{\Omega^0} x$$

$$h^1 = \begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix} = \begin{bmatrix} ReLU(f_1^0) \\ ReLU(f_2^0) \end{bmatrix} = ReLU\left(\begin{bmatrix} f_1^0 \\ f_2^0 \end{bmatrix}\right)$$

$$f^1 = \begin{bmatrix} f_1^1 \\ f_2^1 \end{bmatrix} = \begin{bmatrix} \beta_1^1 + \Omega_{11}^1 h_1^1 + \Omega_{12}^1 h_2^1 \\ \beta_2^1 + \Omega_{21}^1 h_1^1 + \Omega_{22}^1 h_2^1 \end{bmatrix} = \begin{bmatrix} \beta_1^1 \\ \beta_2^1 \end{bmatrix} +$$

$$+ \begin{bmatrix} \Omega_{11}^1 & \Omega_{12}^1 \\ \Omega_{21}^1 & \Omega_{22}^1 \end{bmatrix} \begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix} = \beta^2 + \Omega^2 h^2$$

$$h^2 = \begin{bmatrix} h_1^2 \\ h_2^2 \end{bmatrix} = ReLU\left(\begin{bmatrix} f_1^1 \\ f_2^1 \end{bmatrix}\right) = ReLU(f^1)$$

We choose the regression, so it is a simple sum:

$$f^2 = \beta_0^2 + \Omega_{11}^2 h_1^2 + \Omega_{12}^2 h_2^2 = \beta^2 + \begin{bmatrix} \Omega_{11}^2 & \Omega_{12}^2 \end{bmatrix} \begin{bmatrix} h_1^2 \\ h_2^2 \end{bmatrix} = \beta^2 \Omega^2 h^2$$

Notice: to compute this I have to store plenty parameters and execute the calculus. We have many parameters but imagine with LLMs with billions of parameters. Notice, we can have millions of inputs or outputs.

How will the loss function be?

$S = \{x, y\}$

$L(\Omega, \beta) = \frac{1}{2}(f^2 - y)^2$ (in this case with 2 outputs it is simple, otherwise: $\frac{1}{2}\sum(f^2 - y)^2$)

$L(\Omega, \beta)$ INDICATES THAT L DEPENDS ON ALL ΩS AND βS, WHICH ARE THE PARAMETERS THAT NEED TO BE FIND (x, y, h ARE GIVEN)

Let's compute the gradient: $\frac{\partial L}{\partial \Omega, \beta}$

$$L = \frac{1}{2}(f^2 - y)^2 =$$

$$f^2 = \beta^2 + \Omega^2 h^2 = \beta^2 + \Omega^2 ReLU(f^1) = \beta^2 + \Omega^2 ReLU(\beta^1 + \Omega^1 h^1) = \dots =$$

$= \dots$ KEEP EXPANDING || huge expression

f is a composition of functions, so, in order to get the derivative, we need to compute the derivative of a composition of functions:

$$\frac{\partial (f(g(x)))}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

|| ↑, if you have to compute the gradients, you have to compute also the derivate of each nested function (All or ReLU). Imagine with billions of activation functions... We need to optimize.

To get it, we will use an algorithm that computes the gradient, called BACKPROPAGATION (BACKPROP). || AN ALGORITHM TO COMPUTE THE GRADIENT EFFICIENTLY

$$S = \{(x, y)\}$$

$$L = \frac{1}{2}(f^2 - y^2)^2$$

Notice: I have computed $\frac{\partial L}{\partial f^2} = (f^2 - y)$, when I have to compute: $\frac{\partial L}{\partial \Omega^2_{12}} = \frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial \Omega^2_{12}}$ — I have already computed, so I store always each item that I've calculated and I'll use it in the future

ALWAYS NEEDED

$$\frac{\partial L}{\partial \Omega^2_{11}} = \frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial \Omega^1_{11}} = \frac{1}{2} \cdot 2(f^2 - y) \cdot \frac{\partial f^2}{\partial \Omega^1_{11}}$$

$\frac{1}{2} \cdot 2(f^2 - y)$ is a number

f^2 is computed directly by Ω^2_{11}

in f^2 → DERIVATIVE OF COMPOSITION OF FUNCTIONS

In every partial derivative, $\frac{\partial L}{\partial f^2}$ will always occur.

$$\frac{\partial f^2}{\partial \Omega^1_{11}} = \frac{\partial (\beta^2 + \Omega^2_{11} h^2_1 + \Omega^2_{12} h^2_2)}{\partial \Omega^1_{11}} = \frac{\partial (\Omega^2_{11} h^2_1)}{\partial \Omega^1_{11}} = \Omega^2_{11} \frac{\partial h^2_1}{\partial \Omega^1_{11}} \Rightarrow$$

Il **Backpropagation** (retropropagazione) è l'algoritmo fondamentale utilizzato per calcolare in modo efficiente i gradienti della funzione di perdita (Loss) rispetto a tutti i parametri (pesi e bias) di una rete neurale [1]. Il calcolo dei gradienti è il prerequisito essenziale per ottimizzare la rete usando metodi iterativi come la Discesa del Gradiente o Adam [2] [3].

L'algoritmo si basa sulla **regola della catena (chain rule)** per la derivata di funzioni composte: $\frac{\partial f(g(x))}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$ [1]. Anziché calcolare le derivate analiticamente dall'inizio alla fine (che porterebbe a espressioni gigantesche e ridondanti), il backpropagation procede "all'indietro": calcola prima la derivata alla fine della rete e la propaga verso gli strati iniziali, **riutilizzando i calcoli intermedi** [4].

Esaminiamo nel dettaglio l'esempio pratico riportato nei tuoi appunti, strutturandolo nelle sue due fasi principali: il Forward Pass e il Backward Pass.

1. Il Forward Pass (Passaggio in avanti)

Per poter calcolare le derivate, la rete deve prima elaborare l'input x e memorizzare tutti i valori intermedi calcolati da ogni singolo neurone [5]. Consideriamo la seguente rete composta da un input x, due strati nascosti con funzione di attivazione ReLU e un output lineare:

- **Primo strato nascosto:**
  - Pre-attivazione: $f^0 = \beta^0 + \Omega^0 x = \begin{bmatrix} \beta_1^0 + \Omega_{11}^0 x \\ \beta_2^0 + \Omega_{21}^0 x \end{bmatrix}$ [5]
  - Attivazione: $h^1 = ReLU(f^0) = \begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix}$ [5]
- **Secondo strato nascosto:**
  - Pre-attivazione: $f^1 = \beta^1 + \Omega^1 h^1 = \begin{bmatrix} \beta_1^1 + \Omega_{11}^1 h_1^1 + \Omega_{12}^1 h_2^1 \\ \beta_2^1 + \Omega_{21}^1 h_1^1 + \Omega_{22}^1 h_2^1 \end{bmatrix}$ [5]
  - Attivazione: $h^2 = ReLU(f^1) = \begin{bmatrix} h_1^2 \\ h_2^2 \end{bmatrix}$ [5]
- **Strato di Output:**
  - Predizione finale: $f^2 = \beta_0^2 + \Omega_{11}^2 h_1^2 + \Omega_{12}^2 h_2^2$ [5]
- **Funzione di Loss (Errore Quadratico):**
  - $L = \frac{1}{2}(f^2 - y)^2$ [1] [5]

Tutti i valori intermedi (h_1^1, f_1^1, h_1^2, ecc.) ottenuti per il dato (x, y) vengono salvati in memoria.

2. Il Backward Pass (Il calcolo iterativo del gradiente)

Supponiamo di voler capire come aggiornare un peso specifico situato in profondità nella rete, ad esempio il peso Ω_{11}^1 (che collega h_1^1 a f_1^1). Dobbiamo calcolare la derivata parziale $\frac{\partial L}{\partial \Omega_{11}^1}$ [1].

Invece di espandere l'intera equazione della rete, procediamo a ritroso applicando la regola della catena passo dopo passo:

Step A: Derivata della Loss rispetto all'output Partiamo dalla fine. Come cambia la Loss se cambia l'output f^2? $\frac{\partial L}{\partial f^2} = \frac{1}{2} \cdot 2(f^2 - y) = (f^2 - y)$ [1]

Questa quantità è sempre presente in ogni derivata parziale e rappresenta l'"errore" originario alla fine della rete [1]. Per proseguire, applichiamo la regola della catena: $\frac{\partial L}{\partial \Omega_{11}^1} = \frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial \Omega_{11}^1}$ [1].

Step B: Propagazione attraverso l'output lineare Ora valutiamo come l'output f^2 cambia al variare del nostro parametro Ω_{11}^1. Il parametro Ω_{11}^1 influenza f^2 unicamente passando attraverso il neurone h_1^2 [1]. Quindi: $\frac{\partial f^2}{\partial \Omega_{11}^1} = \frac{\partial (\beta_0^2 + \Omega_{11}^2 h_1^2 + \Omega_{12}^2 h_2^2)}{\partial \Omega_{11}^1} = \Omega_{11}^2 \cdot \frac{\partial h_1^2}{\partial \Omega_{11}^1}$ [1]

Step C: Propagazione attraverso la funzione ReLU Il termine h_1^2 è il risultato di un'attivazione ReLU: $h_1^2 = ReLU(f_1^1)$. La derivata della funzione ReLU rispetto al suo input è la funzione *step* (gradino), che vale 1 se l'input era positivo e 0 se era negativo o nullo [4]. Applicando di nuovo la regola della catena: $\frac{\partial h_1^2}{\partial \Omega_{11}^1} = \frac{\partial ReLU(f_1^1)}{\partial f_1^1} \cdot \frac{\partial f_1^1}{\partial \Omega_{11}^1} = step(f_1^1) \cdot \frac{\partial f_1^1}{\partial \Omega_{11}^1}$ [4]

Step D: Derivata rispetto al peso bersaglio Finalmente siamo arrivati all'equazione in cui risiede il nostro parametro Ω_{11}^1: $f_1^1 = \beta_1^1 + \Omega_{11}^1 h_1^1 + \Omega_{12}^1 h_2^1$ [4] La derivata di questa somma lineare rispetto a Ω_{11}^1 è semplicemente il valore del neurone che lo moltiplica: $\frac{\partial f_1^1}{\partial \Omega_{11}^1} = h_1^1$ [4]

Step E: Assemblare il Gradiente Finale Moltiplicando a ritroso tutti i termini parziali trovati (regola della catena), otteniamo il gradiente esatto per il peso Ω_{11}^1: $\frac{\partial L}{\partial \Omega_{11}^1} = (f^2 - y) \cdot \Omega_{11}^2 \cdot step(f_1^1) \cdot h_1^1$ [4] Noterai che tutti questi elementi ($(f^2 - y)$, Ω_{11}^2, f_1^1, h_1^1) **sono numeri già noti**, perché li abbiamo salvati precedentemente durante il Forward Pass [4].

3. Il vero vantaggio: Il riutilizzo dei calcoli

L'aspetto più potente del backpropagation emerge quando devi calcolare il gradiente per un altro peso nello stesso livello, ad esempio Ω_{12}^1.

Se riscriviamo la catena, avremo: $\frac{\partial L}{\partial \Omega_{12}^1} = \frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial h_1^2} \cdot \frac{\partial h_1^2}{\partial f_1^1} \cdot \frac{\partial f_1^1}{\partial \Omega_{12}^1}$ [4]

Guarda i primi tre termini: $\frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial h_1^2} \cdot \frac{\partial h_1^2}{\partial f_1^1}$. Questa sfilza di derivate vale esattamente $(f^2 - y) \cdot \Omega_{11}^2 \cdot step(f_1^1)$. L'abbiamo **già calcolata** per lo Step E e possiamo semplicemente **riutilizzarla** ("*I can reuse the result of* $\frac{\partial L}{\partial \Omega_{11}^1}$") [4].

Ci basta solo calcolare l'ultimissimo anello della catena ($\frac{\partial f_1^1}{\partial \Omega_{12}^1} = h_2^1$) e moltiplicarlo al blocco precedente: $\frac{\partial L}{\partial \Omega_{12}^1} = [(f^2 - y) \cdot \Omega_{11}^2 \cdot step(f_1^1)] \cdot h_2^1$ [4]

Calcolando a ritroso a ogni iterazione, l'algoritmo non deve mai ricalcolare le stesse derivate parziali più di una volta, permettendo (soprattutto grazie all'uso parallelo delle GPU) di aggiornare milioni di pesi in una frazione di secondo [4].

$$\frac{\partial h_1^2}{\partial w_{11}^1} = \frac{\partial ReLU(f_1^1)}{\partial w_{11}^1} = \frac{\partial ReLU(f_1^1)}{\partial f_1^1} \cdot \frac{\partial f_1^1}{\partial w_{11}^1} = step(f_1^1) \cdot \frac{\partial f_1^1}{\partial w_{11}^1}$$

STEP FUNCTION

$$\frac{\partial f_1^1}{\partial w_{11}^1} = \frac{\partial (b_1^1 + w_{11}^1 h_1^1 + w_{12}^1 h_2^1)}{\partial w_{11}^1} = h_1^1$$

$$\Rightarrow w_{11}^2 \frac{\partial h_1^2}{\partial w_{11}^1} = w_{11}^2 \cdot step(f_1^1) \cdot h_1^1$$

$$\frac{\partial L}{\partial w_{11}^1} = (f^2 - y) \cdot w_{11}^2 \cdot step(f_1^1) \cdot h_1 \quad \text{ALL KNOWN ELEMENTS}$$

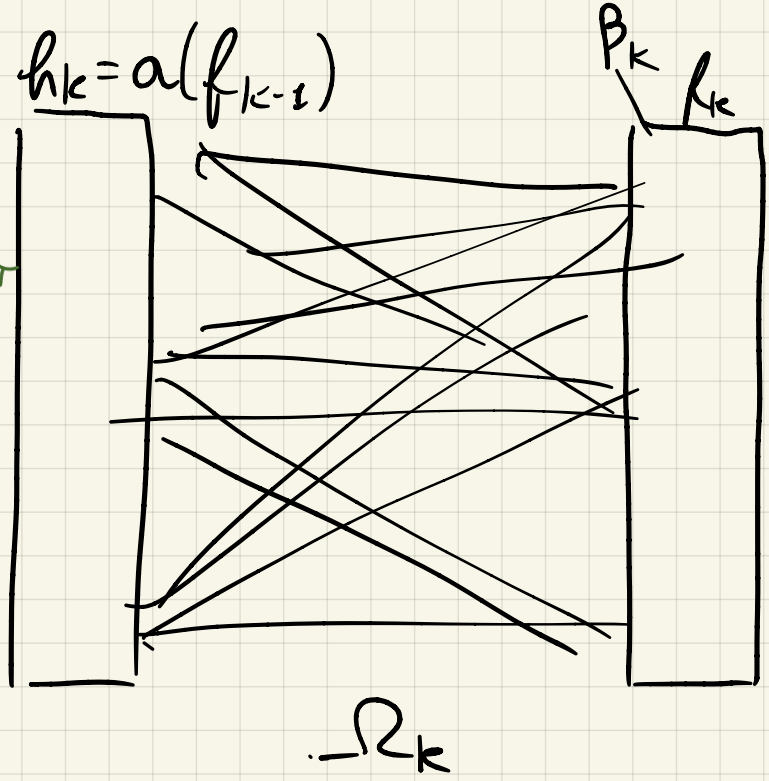
I can reuse the result of $\frac{\partial L}{\partial w_{11}^1}$ w.r.t. $\frac{\partial L}{\partial w_{12}^1}$:

$$\frac{\partial L}{\partial w_{12}^1} = \frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial w_{12}^1} = (f^2 - y) \cdot w_{12}^2 \cdot step(f_1^1) \cdot h_2^1$$

So, BACKPROPAGATION just computes backward at every iteration of gradient descent (in computers, GPU is used because of the possibility to parallelize the computation)

PARAMETER INITIALIZATION: Each time we start with different starting point because of the local minimum

We start by initializing randomly. One very simple way is by a normal distribution ($\mu = 0$, σ is selected for every single matrix, $\sigma_{w_i}^2$; $i = 1, \dots, n$).
Moving to a hidden layer to another, though, can lead to relation problems wrt the importance of our weights in the hidden layers. Let's see an example:



Notice: for each step (Layer-Layer) we choose a DIFFERENT σ^2. YOU WILL SEE IT LATER

Notice:

Notice: all the negative elements will be thrown away by the RELU function

μ, σ^2

μ = x axis position, σ^2 = how wide it is (the integral (Probability) can be max = 1 if $\sigma^2 = 0$ it is a single vertical line, centered in μ, with length 1.

So, if I choose a σ^2 too small the weights Ω can be too much small and every time there is a shift from a hidden layer to another, the value becomes so small, s.t. it vanishes, because of the multiplication chain

Assuming I select some variance $\sigma^2_{\Omega_k}$ s.t. it is very small (i.e. 0,002), so our layers have a very weak relevance in our activation:

$$f_k = \beta_k + \underbrace{\Omega_k h_k}_{\text{THIS IS VERY SMALL}}$$

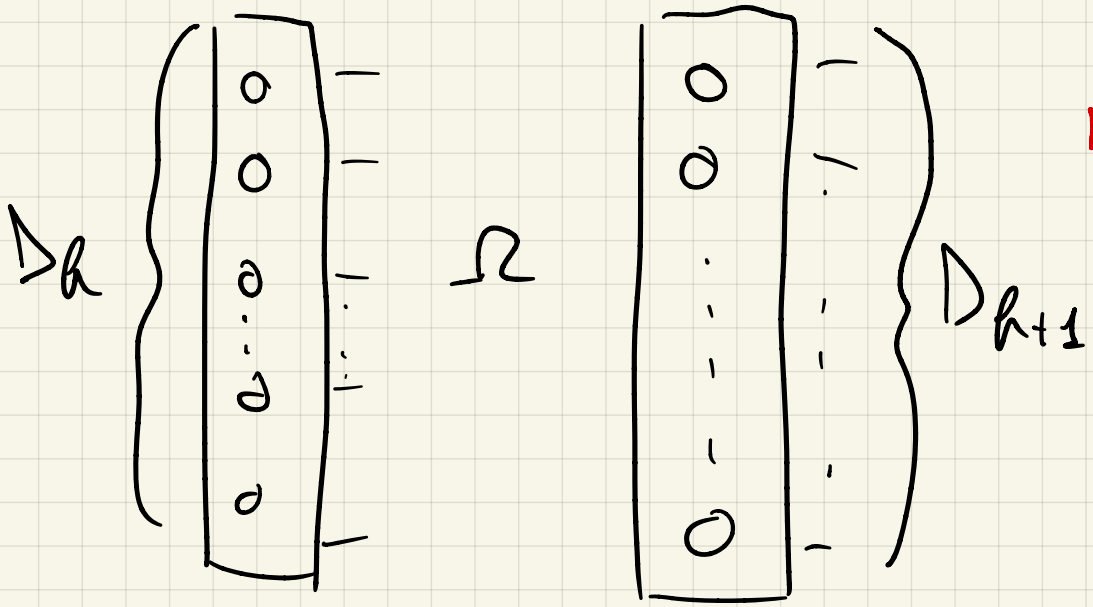
By applying the activation functions, the computation of every hidden layer will lead to tinier and tinier values. So, our convergence by applying gradient will be very slow. Also, if $\sigma^2_{\Omega_k}$ is too large, this will lead to huge values both by computing the algorithm from input to output and viceversa. This leads to overflowes and approximation errors in loss evaluation.

- TOO SMALL PARAMETERS → VANISHING GRADIENT PROBLEM
  - APPROXIMATION ERRORS
  - SLOW CONVERGING

- TOO LARGE PARAMETERS → EXPLODING GRADIENT PROBLEM
  ↓
  - REPRESENTATION ERRORS
  - UNSTABLE LEARNING *

* UNSTABLE LEARNING leads to a not smooth learning of the curve (this could also lead to divergence in extreme cases)

So, to still initialize the matrixes randomly avoiding both vanishing and exploding gradient problems, it is possible to choose a fixed variance:



Where n_h is the number of units of the h-layer

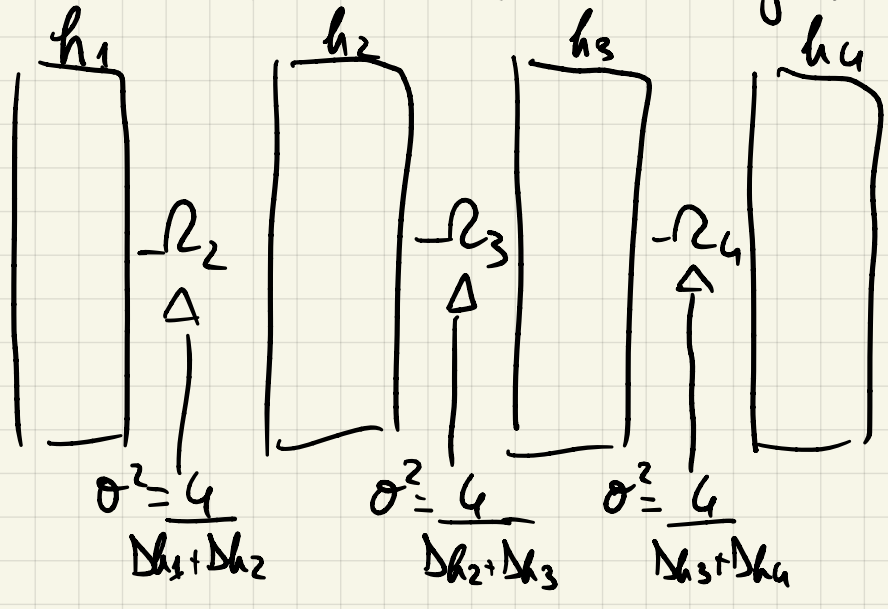
EMPIRICAL RULE

$$\sigma^2_W = \frac{4}{n_h + n_{h+1}}$$

VARIANCE INITIALIZATION

$\sigma^2_W = \frac{2}{n_h}$ FOR 2 LAYERS WITH SAME DIMENSION

Applied to more than 2 hidden layers:



EVERY LIBRARY HAS ITS OWN INITIALIZERS

REGULARISATION: it is a set of techniques which is used to prevent the overfitting

As for linear regression problem, a regularizer is applicable also to NN (both L_1-Lasso and L_2-RIDGE are applicable).

EMPIRICAL RISK — PENALIZATION TERM: Regularization

$$J(\vartheta) = E(\vartheta) + \lambda R(\vartheta)$$

$$R(\vartheta) = \begin{cases} L_2\text{-Ridge} & |\vartheta|^2 \\ L_1\text{-Lasso} & |\vartheta| \\ \text{Elastic Net} & \end{cases}$$

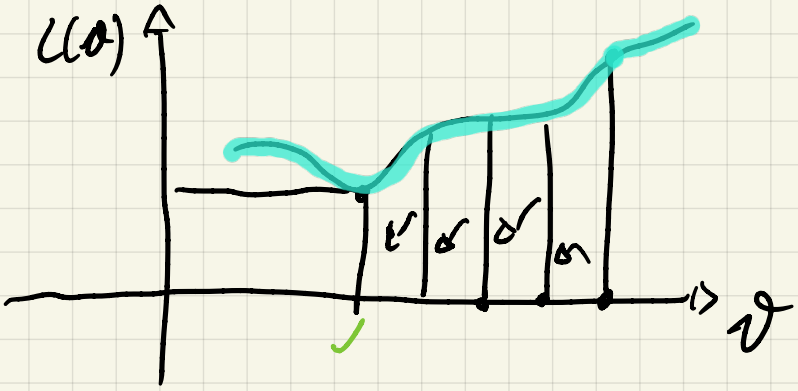
HYPERPARAMETER
the higher λ, the more importance we are giving to $R(\vartheta)$. We are somehow putting the handbrake to him.

$|\vartheta|$ (L_1)

$|\vartheta|^2$ (L_2)

Also in this case, regularization is used to mitigate overfitting (in the network). We now see 2 approaches:

- **EARLY STOPPING**



LOSS FUNCTION REDUCTION BY STOPPING THE TRAINING BEFORE REACHING MINIMUM because we want to avoid overfitting the data

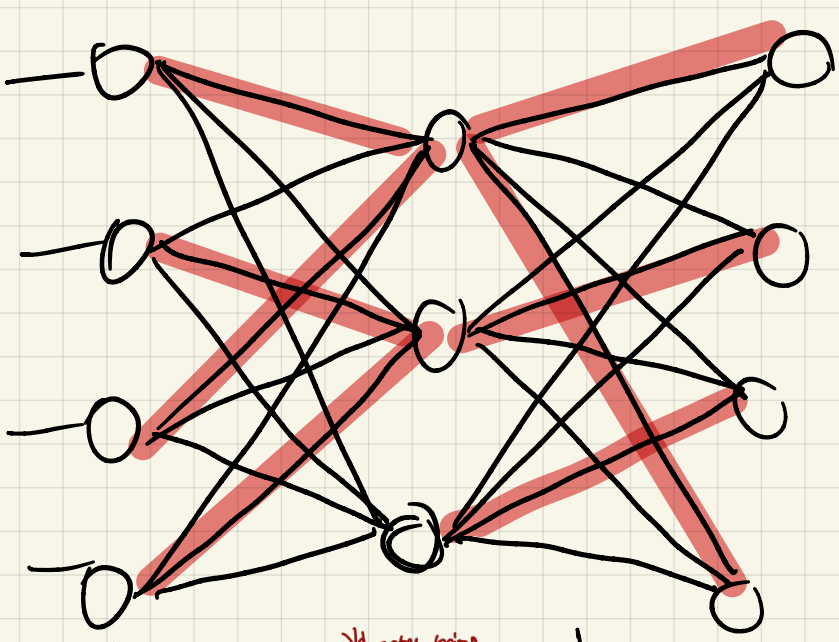
To understand the optimal stop time, validation could be used. The moment to stop is given when validation loss function grows up (it is exactly the moment when training loss is reaching the minimum, so it is going to overfit).

if we overfit our model, by reaching the exact minimum of the loss function we can inspect by ourselves, many times, that "Validation Loss" actually increase in compare to the "Train Loss"

- **DROPOUT**: we are removing some link $\Rightarrow$ setting the weight to 0

The idea is to fix a probability p of removing a link (setting its parameter to 0). At every iteration, some links

are going to drop based on p. Then, at the end of the iteration, the links drop in again and so on until a stopping criteria.



Similar to stochastic gradient descent (add some noise), it is a way to avoid that the network learns "too much" from the training data.

Here regularizers can be combined together.

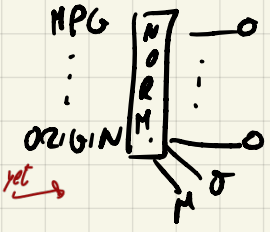
TENSORFLOW LIBRARY TUTORIAL (it is online)

- by Google

TENSORFLOW AND PYTORCH ARE 2 LIBRARIES BASED ON PYTHON USED TO DEFINE A NEURAL NETWORK. BY KNOWING 1 OF THEM, LEARNING THE OTHER ONE IS VERY SIMPLE.

- NORMALIZATION LAYER

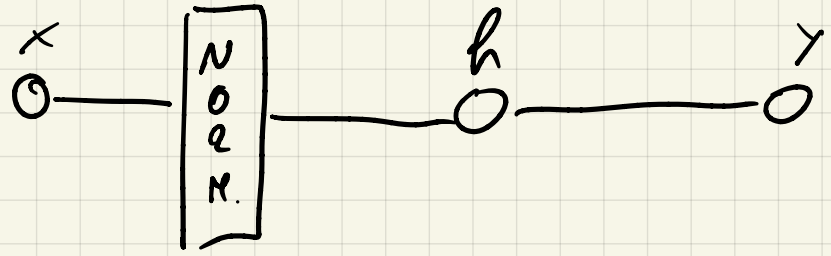
To apply normalization, a layer (not trained) is formed to give as input the normalized train features



$$z = \frac{x - \mu}{\sigma}$$ || the distribution will be centered in 0

- LINEAR REGRESSION

To build the NN, start by linear regression with a single variable as input, "Horsepower".



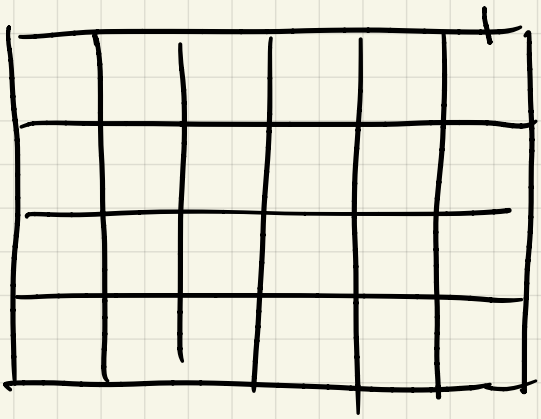
X: "Horsepower" ; h: linear transformer ; y: output

TENSORS

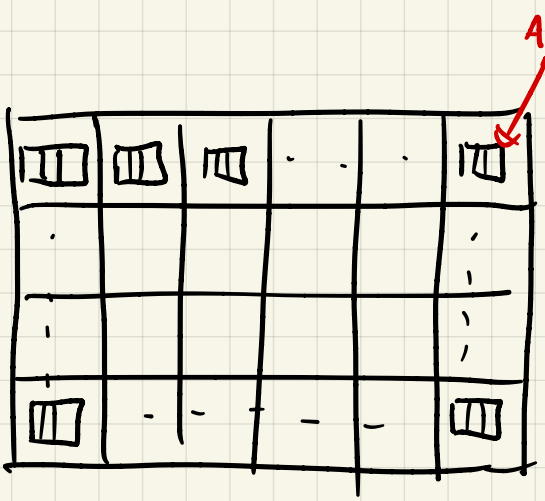
Tensors are an object that generalizes arrays, vectors and matrices to higher dimensions.

EXAMPLE

ARRAY



MATRIX



3D TENSOR

Tensor combine arrays and matrices by creating a structure with higher dimension to abstract objects like images, frames, ecc.